

GAIN-DANN: A Domain-Adversarial Generative Model for Missing Data Imputation in Proteomics

Rita Brito Gama

Thesis to obtain the Master of Science Degree in

Computer Science and Engineering

Supervisor: Prof. Emanuel José Vieira Gonçalves

Examination Committee

Chairperson: Prof. José Luís Brinquete Borbinha
Supervisor: Prof. Emanuel José Vieira Gonçalves
Member of the Committee: Prof. Simon Cai

October 2025

Declaration

I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

Acknowledgments

Para todos os que estão e já não estão.

Seria ingênuo não reconhecer a influência de todos os que estão à minha volta nesta tese. E não apenas no seu culminar, mas no percurso até lá. Dizem que somos um bocadinho de todos aqueles que passam por nós. Pois bem, eu acredito que seja verdade. E a todos dedico esta tese.

Um obrigada à minha família. Um obrigada aos meus amigos. Um obrigada aos meus amigos de faculdade. E um obrigada incansável ao meu orientador, Prof. Emanuel Gonçalves, com quem aprendi bastante ao longo deste ano, não só academicamente, mas também, inevitavelmente, como enfrentar desafios.

Uma ovação geral nunca seria suficiente. Por isso mesmo correndo o risco de soar repetitiva, um obrigada muito especial à minha mãe, que parece saber sempre o que é melhor para mim. Um obrigada especial ao meu pai e minha mãe por todas as oportunidades, amor, carinho, educação e à mistura ética de trabalho. Um obrigada à minha irmã por alinhar em todas as parvoíces, as que vieram e andem vir. Um obrigada aos meus avós que desde sempre me acompanharam. E, mesmo que não possa ser compreendido, um obrigada à Amora, que foi a minha companhia todos os dias desde o início da tese.

E um obrigada aos meus amigos que me acompanham desde o jardim de infância, aos que se juntaram no secundário. Com vocês cresci muito. E aos que agora se juntam na faculdade, sem vocês teria sido mais difícil. E a todos os outros que encontrei por mero acaso.

Um obrigada também aos meus vizinhos que em Lisboa sempre me fizeram sentir em casa.

E um obrigada a todos, estes e outros, que foram uma inspiração, em especial aos meus professores.

A vós!

Abstract

Missing values are one of the most pervasive challenges in proteomics, hiding biological information and reducing the reliability of downstream analysis. Traditional imputation methods, such as missForest, become computational demanding as the size of the dataset grows, making them unfeasible for large-scale proteomics datasets. Generative Adversarial Imputation Network (GAIN) have emerged as a promising alternative to this problem. Previous work in our group extended GAIN to proteomics data, ProtoGain, and demonstrated promising results, laying the foundation for the present study. This work proposes GAIN-DANN, a hybrid model that integrates GAIN with Domain-Adversarial Neural Network (DANN) to improve missing values imputation in proteomics.

The model was designed to exploit shared information across related domains and datasets, aiming to learn domain-invariant representations capable of generalizing to new datasets. In the intra-dataset evaluation, the GAIN-DANN achieved lower RMSE than the baseline ProtoGain while simultaneously mitigating batch effects, suggesting that the adversarial nature of the model enabled it to focus on biological rather than domain-specific variation. In the cross-dataset setting, where pretrained representations were reused across datasets, the GAIN-DANN reduced the computational time by approximately 75% and achieved a substantial lower RMSE than ProtoGain models trained from scratch. These results demonstrated that previously learned representations can be successfully reused to improve imputation in smaller datasets. Overall, the results highlight the potential of combining adversarial domain adaptation with generative imputation to enhance missing values imputation.

Keywords

Generative Adversarial Imputation Network, Domain-Adversarial Neural Network, Domain Adaption, Missing Values Imputation

Resumo

Missing values são um dos desafios mais recorrentes em proteômica, escondendo informação biológica e reduzindo a fiabilidade das análises subsequentes. Métodos tradicionais de imputação, como o missForest, tornam-se computacionalmente exigentes à medida que o tamanho do dataset aumenta, tornando-se inviáveis para estudos de proteômica em larga escala. As Generative Adversarial Imputation Networks (GAIN) surgem como uma alternativa promissora. Trabalhos anteriores no nosso grupo expandiram o GAIN para dados proteômicos, ProtoGain, e demonstraram resultados promissores, estabelecendo as bases para o presente estudo. Este trabalho propõe o GAIN-DANN, um modelo híbrido que integra o GAIN com Domain-Adversarial Neural Networks (DANN) para melhorar a imputação de missing values em proteômica.

O modelo foi concebido para explorar informação partilhada entre domínios e datasets relacionados, com o objetivo de aprender representações invariantes ao domínio, capazes de generalizar para novos datasets. Na avaliação intra-dataset, o GAIN-DANN apresentou RMSE inferior ao modelo de referência ProtoGain, mitigando simultaneamente os batch effects, sugerindo que a componente adversarial permitiu ao modelo focar-se em variações biológicas em vez de específicas do domínio. No cenário cross-dataset, onde representações pré-treinadas foram reutilizadas entre datasets, o GAIN-DANN reduziu o tempo computacional em aproximadamente 75% e apresentou RMSE substancialmente inferior aos modelos ProtoGain treinados de raiz. Estes resultados demonstram que representações previamente aprendidas podem ser reutilizadas com sucesso para melhorar a imputação em datasets menores. No geral, os resultados destacam o potencial de combinar adaptação adversarial de domínio com im-

putação generativa para melhorar a imputação de missing values.

Palavras Chave

Generative Adversarial Imputation Network, Domain-Adversarial Neural Network, Adaptação de Domínio, Imputação de Missing Values

Contents

1	Introduction	1
1.1	Motivation	2
1.2	Work Objectives	3
1.3	Contributions	4
1.4	Document Outline	4
2	Background Research	5
2.1	Proteomics	5
2.2	Missing Data	6
2.3	Generative Adversarial Networks	7
2.4	Transfer Learning	8
2.5	Graph Neural Network	10
2.5.1	Graph Convolutional Networks	12
2.5.2	Link prediction on GNNs	13
2.5.3	Adversarial Training on GNNs	16
3	Related Work	19
3.1	Generative Adversarial Imputation Networks	19
3.2	ProtoGain	20
3.3	Domain-Adversarial Neural Networks	21
4	The GAIN-DANN Model	25
4.1	Model Design	26
4.1.1	Input	26
4.1.2	Architecture	27
4.2	Training	31
4.3	Evaluation	37
4.4	Inference	37
4.5	Hyperparameters Optimization	38
4.6	Imputation with a pretrained model	39

4.6.1	Full Encoder Freezing	40
4.6.2	Partial Encoder Freezing	40
4.6.3	Domain-Adversarial Adaptation	40
4.7	Implementation and Usage Details	40
5	Results and Discussion	43
5.1	Datasets	43
5.1.1	HeLa dataset	44
5.1.2	PRIDE datasets	48
5.2	Evaluation	50
5.2.1	Intra-dataset	50
5.2.2	Cross-Dataset	56
5.3	Discussion	59
5.3.1	Intra-dataset	60
5.3.2	Cross-dataset	60
5.3.3	Computation considerations	61
6	Conclusion	63
6.1	Conclusions	63
6.2	Limitations and Future Work	64
	Bibliography	65
A	Additional Figures and Tables	71

List of Figures

4.1	GAIN-DANN architecture	35
5.1	Distribution of protein intensities HeLa dataset	44
5.2	Missingness across samples and proteins in the HeLa dataset	46
5.3	Protein intensity distributions for low-missingness and high-missingness proteins in the HeLa dataset	46
5.4	Missingness per HeLa project	47
5.5	UMAP projection of HeLa	48
5.6	Protein intensities on Brain sample tissues	50
5.7	UMAP projection of HeLa-Reduced	52
5.8	Distribution of protein intensities HeLa-Reduced	52
5.9	Missing values per HeLa-Reduced project	54
5.10	Imputation quality HeLa-Reduced	55
5.11	Correlation sample imputed vs. sample observed HeLa-Reduced	55
5.12	Coverage similarity	57
5.13	UMAP projection of HeLa-Reduced-NoPXD042233	57
5.14	Missingness in HeLa-Reduced-NoPXD042233	58
A.1	Protein intensities distribution by HeLa project	72

List of Tables

4.1	Optimal Hyperparameters	39
5.1	Number of samples per HeLa project	47
5.2	PRIDE datasets	49
5.3	PRIDE datasets restricted to a single tissue type	49
5.4	Protein intensities distributions per HeLa-Reduced project	53
5.5	Comparison between GAIN-DANN and ProtoGain in RMSE and runtime	59
A.1	PRIDE datasets with multiple tissues	73

List of Algorithms

4.1	Stochastic gradient descent training procedure of GAIN-DANN	36
-----	---	----

Acronyms

DANN	Domain-Adversarial Neural Network
DNN	Deep Neural Network
GAIN	Generative Adversarial Imputation Network
GAN	Generative Adversarial Network
GNN	Graph Neural Network
GRL	Gradient Reversal Layer
MS	Mass spectrometry
MAR	Missing at random
MCAR	Missing completely at random
MNAR	Missing not at random
PPI	Protein-Protein Interaction
RMSE	Root Mean Square Error

1

Introduction

Contents

1.1 Motivation	2
1.2 Work Objectives	3
1.3 Contributions	4
1.4 Document Outline	4

In the early 1950s, Frederick Sanger determined the first complete sequence of a protein, the insulin, marking a groundbreaking milestone. By the late 1960s, numerous proteins had been sequenced paving the way for systematic studies of protein structure and function. Margaret Dayhoff, often regarded as one of the founders of bioinformatics, developed computational methods for comparing protein sequences as well as the first protein sequence database, which laid the foundation for systematic cataloging protein sequences. Today, it is estimated that the human genome contains approximately 20,000 protein-coding genes, but the number of resulting proteoforms is thought to be vastly greater due to processes such as splice variants and post-translational modifications [1].

Over the years, advances in sequencing technologies have significantly increased the power of proteomics, the large-scale study of proteins, making possible the generation of large-scale datasets. However, these sequencing methods are not immune to technological limitations. Missing values in pro-

teomics data are a common occurrence, often arising from limitations in sensitivity, detection thresholds, or variability in experimental conditions. Furthermore, the inherent complexity of proteomics contributes to this issue, where the expression of proteins can vary widely depending on environmental factors and cell type [2].

Proteomics plays a pivotal role in drug development and biomarker discovery, particularly biomarkers in cancer [3]. However, a critical challenge in proteomics lies in addressing missing values within datasets, which can severely affect the reliability of downstream analysis. This is particularly problematic when using traditional machine learning, which often require complete datasets to perform effectively.

1.1 Motivation

Deep learning methods have shown to be more robust to missing data due to their ability to handle incomplete datasets. Nevertheless, the presence of missing values still represents a challenge, as it may require additional preprocessing steps, such as data imputation, to ensure optimal performance and accuracy. Thus, addressing this challenge is crucial to fully leverage the potential of proteomics data, especially in drug and biomarker discovery [4].

Traditional imputation methods, such as missForest, have been extensively used in missing values imputation. However in recent years, deep learning approaches have gained significant attention across numerous fields, including data imputation, leading to notable advancements in performance and accuracy. Among these approaches, Generative Adversarial Network (GAN) have demonstrated remarkable potential in generating realistic synthetic data. Yoon et al. introduced Generative Adversarial Imputation Networks [5], an extension of GANs specifically designed to address missing data, which has shown superior performance compared to traditional methods like missForest. Moreover, [6] demonstrated that GAIN outperformed missForest on real-life proteomics datasets.

Despite their success, training GANs and GAINs on large proteomics datasets is computationally demanding, as they require learning a substantial number of parameters. Moreover, training from scratch every new dataset is often inefficient and may not yield optimal performance, especially when dealing with small datasets that lack sufficient samples for generalization. These constraints motivate the use of transfer learning, which enables models trained on larger or related datasets to be fine-tuned and adapted to new datasets with limited data [7, 8].

A central motivation for the proposed approach is the assumption that while a protein may be undetected in one dataset, it could be present in another dataset, since the missingness can often be caused by the stochastic nature of mass-spectrometry, limitations in sensitivity, detections thresholds and/or different experimental setups. As such, the missingness may not reflect true biological information. This is where transfer learning arises as a promising avenue. A model trained on a dataset where

a given protein is observed, should, in principle, be able to leverage the knowledge learned to impute its abundance in datasets where the given protein is missing. In addition, most machine learning models require substantial amounts of data to learn robustly, as such the ability to leverage learned representation from large-scale datasets becomes particularly valuable in proteomics. Building on the promising results of GAIN [5, 6] and ProtoGain¹ this work explores the integration of transfer learning to enhance the imputation of missing values in proteomics datasets.

Additionally, the concept of leveraging knowledge from Protein-Protein Interaction (PPI) networks to estimate protein abundance in a sample also presents itself as a promising approach for addressing incomplete data in proteomics. By harnessing the large corpus of biological information in protein-protein interaction databases, such as STRING², it may be possible to infer missing relationships within a dataset, thereby improving data completeness. From a graph-theoretic point of view, this is usually framed as a link prediction problem, where the objective is to predict the likelihood of a missing or unobserved edge between nodes in the network.

Finally, this work is conducted in close collaboration with the PRIDE³ proteomics database (EMBL-EBI, Cambridge, UK), ensuring access to high-quality proteomics data for the development and validation of the proposed methodologies [9]. In addition, this research builds upon the ongoing efforts of the Quantitative Biology group at INESC-ID, particularly on the application of GAIN to proteomics data [6]. This thesis was also developed in parallel with a complementary thesis focused on the deployment of the resulting models as a public accessible tool.

1.2 Work Objectives

Despite significant advances in proteomics, challenges persist in effectively handling missing values. These challenges are further compounded by the intrinsic nature of proteomics data, where protein expression can vary substantially across different biological conditions.

Generative Adversarial Imputation Networks have shown itself as a promising avenue to address these limitations. However, the transferability of the knowledge learned from one proteomics dataset to another remains unexplored. This motivated the development and evaluation of a model that combines the strengths of GAIN with adversarial domain adaptation, constituting the main objective of this work. To this end, a GAIN-DANN model was proposed, integrating GAIN with Domain-Adversarial Neural Networks to learn domain-invariant representations and, therefore, enhance imputation quality across diverse proteomics datasets.

¹<https://github.com/QuantitativeBiology/GenerativeProteomics>

²<https://string-db.org/>

³<https://www.ebi.ac.uk/pride/>

1.3 Contributions

The main contributions of this work can be summarized as follows:

- The development of GAIN-DANN framework, a hybrid model that leverages the strengths of generative imputation, through GAIN, and domain adaptation, which allows learning domain-invariant representations of the data, that can be use to knowledge transfer across related datasets;
- By learning a domain-invariant representation of the data, the model can generalize across related domains and mitigate batch effects if present;
- Implementation of fine-tuning strategies that enable the imputation of missing values by leveraging previously learned representations;
- Extensive evaluation of the proposed model in both intra-dataset and cross-dataset settings, including benchmarking its performance against the ProtoGain model;
- Empirical evidence that pretrained representations can help improve imputation quality while substantially reducing computational time compared to training a ProtoGain model from scratch, which does not leverage pretrained representations;
- Demonstrated that small datasets with moderate missingness can benefit from prior knowledge acquired from other datasets, highlighting the model ability to generalize across datasets.

1.4 Document Outline

The document is divided into six chapters. Chapter 2 covers the fundamental concepts of proteomics, missing data, GANs, transfer learning and Graph Neural Network (GNN) which together form the theoretical basis of this work. Chapter 3 reviews existing approaches based on GAIN, namely ProtoGain, as well as the Domain-Adversarial Neural Network model, a promising architecture for transfer learning. The proposed GAIN-DANN model is described in detail in Chapter 4, which details its architecture, training strategy and evaluation methodology. Additionally, the fine-tuning strategies developed to enhance the imputation of missing values using a pretrained GAIN-DANN model are also described. The results are discussed in Chapter 5, including descriptions of the datasets used. Finally, Chapter 6 summarizes the main findings and discusses its current limitation and potential future work.

2

Background Research

Contents

2.1 Proteomics	5
2.2 Missing Data	6
2.3 Generative Adversarial Networks	7
2.4 Transfer Learning	8
2.5 Graph Neural Network	10

This section introduces some key concepts relevant to the scope of our work. The first two sections establish the importance of proteomics and provide context for the missing data problem, discussed in Sections 2.1 and 2.2, respectively. These are followed by detailed descriptions of the core concepts of our approach: Generative Adversarial Networks in Section 2.3, and Transfer Learning in Section 2.4. Finally, Section 2.5 provides an overview of Graph Neural Networks.

2.1 Proteomics

Proteomics, the large-scale study of proteins, is central to understanding the molecular mechanisms underlying biological systems [10]. Proteins are the primary executors of biological processes, driv-

ing synthetic, catalytic, and regulatory functions within cells. They interact in complex networks and dynamically adapt to internal and external perturbations, thereby defining a cell's functional state and phenotype [11]. Understanding the proteome, a collection of proteins that perform their functions at specific times and locations in the cell, is crucial for advancing biological research. In translational medicine, proteins that consistently change in abundance in relation to a disease phenotype are particularly valuable as potential biomarkers for the phenotype in clinical diagnosis and therapeutic interventions [11].

An important outcome of in-depth proteomics studies has been the discovery that diverse cellular systems often share highly similar proteomes, with relatively few proteins being uniquely detectable under specific conditions [11]. This finding, supported by the Human Protein Atlas, highlights that the identity of cells and tissues is primarily determined by the abundance at which they express their constituent proteins, rather than the presence or absence of certain proteins [11].

Mass spectrometry (MS) has been revolutionary in enabling the exploration of the proteome. A mass spectrometer, comprising an ion source, a mass analyzer, and a detector, measures the mass-to-charge ratio of ionized analytes with unparalleled precision [10]. Modern MS-based techniques can identify, quantify, and analyze nearly any protein expressed in a biological sample, as well as detect post-translational modifications (PTMs) and characterize protein-protein interactions [11]. These capabilities have established MS as an indispensable tool for proteomics, offering insights into the composition, structure, function and dynamics of proteins in cellular contexts [10, 11].

Despite its transformative impact, proteomics continues to face significant challenges. Cellular proteomes are highly complex, with many proteins expressed at low abundances that require sensitive and precise analytical techniques for detection [10]. Furthermore, a comprehensive and quantitative understanding of the proteome, including its structure, interactions and functional dynamics, remains a fundamental challenge in biology [11]. Nevertheless, the ongoing advancements in MS technology, including improved resolution, mass accuracy and dynamic range, are progressively overcoming these limitations and expanding the horizons in both research and clinical applications [10, 11].

2.2 Missing Data

Missing values can be categorized by three main missing mechanisms, based on the relationship between the missing data and the observed data [12]. These categories are:

- **Missing completely at random (MCAR):** The probability of an observation being missing is entirely independent of both observed and unobserved data. In this case, the missing data introduces no bias.
- **Missing at random (MAR):** The probability of an observation being missing is related only to the

observed data and is independent of the unobserved data. Consequently, the missing data can be fully explained and accounted for by the observed data.

- **Missing not at random (MNAR):** The probability of an observation being missing is related to both observed and unobserved data [13].

Missing values in proteomics, often referred to as missing proteins, represent a pervasive issue that can significantly impact downstream analysis. These missing values typically fall into two categories: inconsistency and coverage [14]. Inconsistency missingness occurs when a protein is detected in at least one sample. This type of missingness is a regular occurrence not only in proteomics but also in other high-dimensional high-throughput technologies such as RNA-sequencing. Dealing with inconsistency requires the application of missing value imputation methods to recover the missing data. In contrast, coverage missingness represents a more complex challenge, occurring when a protein is absent across all samples despite its expected presence. While some proteins may be legitimately absent in the sample, others remain undetected due to several factors, such as biases in data acquisition, limitations in instrument sensitivity (e.g. protein abundances below the instrument detection limit), and the high biological complexity [14]. Dealing with coverage missingness typically requires integrating observed data with external knowledge, such as reference networks, to infer the presence of proteins.

In proteomics, it is widely believed that MAR and MCAR are the same. In contrast, MNAR occurs in situations where the likelihood of missingness depends on the value of the variable itself. For example, low-abundance proteins are more likely to be missing, specifically when it is close to the limit of detection of the instrument. MAR, on the other hand, arises from abundance-independent factors, such as random fluctuations or technical noise. However, most likely both types, MNAR and MAR, will be present simultaneously, making it difficult to isolate their contributions, and, thus, complicating a proper imputation [14].

2.3 Generative Adversarial Networks

Generative Adversarial Networks have exhibited remarkable capability in synthesizing highly realistic data [15]. A GAN consists of two adversarial networks: a generator G and a discriminator D . Goodfellow et al. describe the intuition behind this architecture as analogous to a dynamic between a team of counterfeiters and a team of police officers. The counterfeiters aim to produce fake currency that the police cannot detect, while the police attempt to identify the counterfeits [15]. This analogy effectively captures the dynamic of GANs, where the generator seeks to deceive the discriminator by generating highly realistic data, such that the discriminator cannot distinguish the generated data, p_g , from the real data distribution, p_{data} . This interplay represents a minimax two-player game with the value function $V(G, D)$. The adversarial objective is formalized in Equation 2.1.

$$\min_G \max_D V(G, D) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))] \quad (2.1)$$

The success of GANs lies in adversarial training, where the generator and discriminator are trained simultaneously. To mitigate overfitting, the training alternates between k steps of optimizing the discriminator and one step of optimizing the generator. Ideally, in the end, the generator produces data indistinguishable from the real data, leading the discriminator's output to converge to $D(x) = \frac{1}{2}$.

Building on the GAN architecture, Generative Adversarial Imputation Nets, extend the approach for missing data imputation [5]. GAIN significantly outperforms state-of-the-art imputation methods, such as missForest and Auto-encoder, across various experimental settings. These settings include varying missing rates (up to 80%), number of samples (up to 4×10^4), and number of feature dimensions (up to 25).

The fundamental principle of GAIN is analogous to that of GANs, but it adapts the adversarial framework specifically to handle missing data. The generator G learns to predict both the observed and missing components of a real data vector, while the discriminator D attempts to identify which components are observed and which are imputed. To achieve this, the input vector is coupled with a mask matrix M , which records the observed and missing parts of the data and is fed into the generator.

GAIN introduces two distinct loss functions for the generator: $\mathcal{L}_G$, representing the error associated with predicting missing components, and $\mathcal{L}_M$, reflecting the error in reconstructing observed components. Additionally, to guide the discriminator, a hint matrix is provided, supplying partial information about the observed and imputed parts. This structure helps the generator align its predictions with the true underlying data distribution, thus enhancing imputation performance.

2.4 Transfer Learning

Transfer learning leverages knowledge from specific domains or tasks and applies it to similar or related ones. This approach proves particularly valuable when dealing with insufficient training data, a challenge that is almost inevitable in certain research fields [7, 8]. Bioinformatics, for example, suffers from the challenges of costly data acquisition and annotation, making the construction of large-scale, well-annotated datasets prohibitively expensive [8].

Although conceptually straightforward, its practical application can be challenging. Its success rests on two key principles. First, unlike traditional machine learning methods, transfer learning relaxes the hypothesis that the training and test data must share the same feature space and distribution [8]. This flexibility allows models to be more easily adapted to new tasks or domains. Second, transfer learning mitigates the need for extensive data labeling efforts, as it enables knowledge to be transferred from one or more source domains to the target domain, thereby reducing both training data requirements and

training time.

For example, learning to recognize apples may aid in recognizing pears, illustrating how knowledge gained in one domain can benefit related tasks [7]. Additionally, this transfer can even introduce additional knowledge that may not exist within the limited target data, further enhancing the performance and robustness of the model [8].

Studies such as [7, 8, 16, 17] have demonstrated that transfer learning methods consistently outperform traditional non-transfer learning approaches, particularly in scenarios with limited training data or where knowledge can be effectively transferred across related domains.

A comprehensive definition of transfer learning, synthesizing key elements from [7, 8], along with the necessary foundational concepts is provided as follows:

A domain $\mathcal{D}$ is defined as $\mathcal{D} = \{\mathcal{X}, P(X)\}$ where $\mathcal{X}$ represents the feature space, and $P(X)$ is the marginal probability distribution over $X = \{x_1, \dots, x_n\} \in \mathcal{X}$. A task $\mathcal{T}$ is defined as $\mathcal{T} = \{y, f(x)\}$, where y represents the label space and $f(x)$ is the target predictive function, which can also be viewed as the conditional probability $P(y | x)$.

Definition 1. Transfer Learning. Given a target task $\mathcal{T}_t$ and a target domain $\mathcal{D}_t$, transfer learning aims to improve the learning of the target predictive function $f_{\mathcal{T}}(\cdot)$ by transferring latent knowledge from a source domain $\mathcal{D}_s$ and a source task $\mathcal{T}_s$. This transfer occurs under the condition that $\mathcal{D}_s \neq \mathcal{D}_t$ and/or $\mathcal{T}_s \neq \mathcal{T}_t$. In most cases, the size of $\mathcal{D}_s$, denoted as N_s , is significantly larger than the size of $\mathcal{D}_t$, denoted as N_t , such that $N_s \gg N_t$.

In recent years, deep learning has become an integral part of transfer learning, following a broader trend seen across many other fields. Tan et al. [8] define deep-transfer learning as follows:

Definition 2. Deep Transfer Learning. A deep transfer learning task is defined by the tuple $\langle \mathcal{D}_s, \mathcal{T}_s, \mathcal{D}_t, \mathcal{T}_t, f_{\mathcal{T}}(\cdot) \rangle$, where $f_{\mathcal{T}}(\cdot)$ is a nonlinear function implemented using a deep neural network.

In simpler terms, transfer learning enhances the performance of a model by leveraging knowledge from source domains and/or tasks to address challenges in the target domain or task. However, the model must still adapt this transferred knowledge to the specific target domain or task.

Tan et al. [8] categorizes deep transfer learning into four categories:

- **Instances-based deep transfer learning** reuses some instances from the source domain that are similar to those in the target domain. These selected instances are then added to the training set of the target domain, with appropriate weight values assigned to them to reflect their relevance.
- **Mapping-based deep transfer learning** involves mapping instances from both the source and target domains into a new feature space with more similarity. These methods are based on the

assumption that, although instances in the two domains may differ, they can become more similar when represented in a carefully constructed feature space. Essentially, these methods aim to shape the space in which the domain disparity between the two domains is reduced, allowing for the use of both domains.

- **Network-based deep transfer learning** reuses a partial network pre-trained in the source domain, including its network structure and connection parameters, to be a part of a deep neural network used in the target domain.
- **Adversarial-based deep transfer learning** involves incorporating adversarial techniques inspired by the Generative Adversarial Nets [15], to identify transferable representations that are applicable to both the source and target domain. It is based on the assumption that, for effective transfer, a good representation should be discriminative for the primary learning task while being indistinguishable between the source and target domains.

2.5 Graph Neural Network

While neural networks can effectively capture hidden patterns in Euclidean data, Graph Neural Networks first proposed by Scarselli et al. have gained significant traction in recent years thanks to their ability to operate over graph-structured data (non-Euclidean data) [18]. GNNs are specifically designed neural networks that iteratively update node representations by aggregating the representations of neighboring nodes and their own representation from previous iterations. This iterative process allows GNNs to embed both the inherent structure and preserve the semantic information of a graph. Consequently, GNNs have become powerful tools in diverse applications, including social networks, e-commerce and biological networks [19].

The goal of the GNN model is to learn a state embedding $x_n \in \mathbb{R}^s$, which encodes the information of the neighborhood for each node [18]. The state embedding x_n is a s -dimensional vector representation of node n and can be used to produce some desirable output o_n , such as a node label. GNNs rely on two primary functions, the *local transition function*, f , that computes the state embedding, and the *local output function*, g , which describes how the output is produced. The state x_n and output o_n of node n are described by Equations 2.2a and 2.2b, respectively.

$$\mathbf{x}_n = f_{\mathbf{w}}(\mathbf{l}_n, \mathbf{l}_{co[n]}, \mathbf{x}_{ne[n]}, \mathbf{l}_{ne[n]}) \quad (2.2a)$$

$$\mathbf{o}_n = g_{\mathbf{w}}(\mathbf{x}_n, \mathbf{l}_n) \quad (2.2b)$$

For a given node n , $co[n]$ and $ne[n]$ are the set of edges and neighboring nodes connected to n ,

$\mathbf{l}_n, \mathbf{l}_{co[n]}, \mathbf{x}_{ne[n]}, \mathbf{l}_{ne[n]}$ represent the labels of n , the labels of its edges, the states and the labels of the nodes in the neighborhood of n , respectively. Both f_w and g_w can be interpreted as feedforward neural networks.

Let $\mathbf{x}, \mathbf{l}, \mathbf{o}, \mathbf{l}_N$ be the vectors constructed by stacking all states, all labels, all outputs, and all node labels, respectively. Then 2.2 can be written in a compact form as:

$$\begin{aligned}\mathbf{x} &= F_{\mathbf{w}}(\mathbf{x}, \mathbf{l}) \\ \mathbf{o} &= G_{\mathbf{w}}(\mathbf{x}, \mathbf{l}_N)\end{aligned}\tag{2.3}$$

$F_{\mathbf{w}}$ represents the *global transition function* and $G_{\mathbf{w}}$ the *global output function*, which are stacked versions of f_w and g_w , respectively, for all nodes in the graph.

It is possible to derive that the value of $\mathbf{x}$ corresponds to the fixed point of equation 2.3, provided that $F_{\mathbf{w}}$ is a contraction map. Banach's fixed point theorem [18] suggests that the states can be computed iteratively as follows:

$$\mathbf{x}(t+1) = F_{\mathbf{w}}(\mathbf{x}(t), \mathbf{l})\tag{2.4}$$

Here, $\mathbf{x}(t)$ denotes the t -th iteration of $\mathbf{x}$. The dynamical system 2.4 converges exponentially fast to the solution of 2.3 for any initial value of $\mathbf{x}(0)$.

In GNNs, learning involves estimating the parameter $\mathbf{w}$ such that the function φ_w accurately represents the data in the learning datasets, which is defined in Equation 2.5.

$$\begin{aligned}\mathcal{L} &= \{(\mathbf{G}_i, n_{i,j}, \mathbf{t}_{i,j}) | \mathbf{G}_i = (\mathbf{N}_i, \mathbf{E}_i) \in \mathcal{G}; \\ &\quad n_{i,j} \in \mathbf{N}_i; \mathbf{t}_{i,j} \in \mathbb{R}^m, 1 \leq i \leq p, 1 \leq j \leq q_i\}\end{aligned}\tag{2.5}$$

Here, q_i represents the number of supervised nodes in G_i . For graph-focused tasks, a single node is typically designated as the target ($q_i = 1$), while, for node-focused tasks, supervision can theoretically be applied to all nodes. The learning objective can be expressed as the minimization of a quadratic cost function (Equation 2.6).

$$e_{\mathbf{w}} = \sum_{i=1}^p \sum_{j=1}^{q_i} (t_{i,j} - \varphi_w(G_i, n_{i,j}))^2\tag{2.6}$$

The learning algorithm is based on a gradient-descent strategy and is composed of the following steps:

1. The states $\mathbf{x}_n(t)$ are iteratively updated according to Eq. 2.4 until at time T they approach the fixed point solution of Eq. 2.3: $\mathbf{x}(T) \approx \mathbf{x}$.
2. The gradient $\frac{\partial loss}{\partial \mathbf{w}}$ is computed.

3. The weights w are updated accordingly to the gradient computed in the previous step.

2.5.1 Graph Convolutional Networks

Convolutional Neural Networks (CNNs) have become one of the most powerful tools in deep learning, especially in the fields of computer vision. These networks leverage convolutional operations, which involve applying filters to the input data, usually grid-like data, to capture local patterns and features. These operations allow CNNs to automatically learn spatial hierarchies of features, making them highly effective for tasks like image classification and object detection. The success of CNNs has inspired similar approaches for graph-structured data, leading to the development of Convolutional Graph Neural Networks (ConvGNNs) [20].

Convolutional Graph Neural Networks aim to extend the concept of convolution operation to graphs, these are divided into two main categories, **spectral-based** approaches and **spatial-based** approaches [20]. The main distinction between these approaches is explained succinctly in [20]. While spectral-based approaches define graph convolutions by introducing filters inspired by graph signal processing, spatial-based approaches draw inspiration from Recurrent Graph Neural Networks defining graph convolutions through information propagation.

The spectral graph theory considers an undirected graph. This graph is mathematically represented by the normalized graph Laplacian matrix $\mathbf{L} = \mathbf{I}_n - \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}$, where $\mathbf{A}$ is the adjacency matrix and $\mathbf{D}$ the degree matrix, with $\mathbf{D}_{ii} = \sum_j (\mathbf{A}_{i,j})$. The Laplacian matrix is a real symmetric positive semidefinite matrix. Thus, the Laplacian matrix can be factored as $\mathbf{L} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^T$ where $\mathbf{U}$ represents the matrix of eigenvectors, $\mathbf{U} = [\mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{n-1}] \in \mathbb{R}^{n \times n}$ and $\mathbf{\Lambda}$ the diagonal matrix of the eigenvalues, $\mathbf{\Lambda}_{ii} = \lambda_i$.

Before proceeding to the formulation of the graph convolution of the ConvGNNs, it is necessary to first establish some definitions and concepts.

Drawing inspiration from graph signal processing, a graph signal $\mathbf{x} \in \mathbb{R}^n$ is a feature vector of all nodes of a graph, where x_i denotes the value of the i -th node. The graph Fourier transform applied to a signal $\mathbf{x}$ is defined as $\mathcal{F}(\mathbf{x}) = \mathbf{U}^T \mathbf{x}$, as such the inverse graph Fourier transform is given by $\mathcal{F}^{-1}(\hat{\mathbf{x}}) = \mathbf{U} \hat{\mathbf{x}}$, with $\hat{\mathbf{x}}$ the resulted signal from the graph Fourier transform.

As such, the graph Fourier transform maps the input graph signal onto an orthonormal space, where the basis of the space is defined by the eigenvectors of the normalized graph Laplacian. Thus, $\hat{\mathbf{x}}$, the transformed signal, represents the coordinates of the graph signal in the orthonormal basis. Accordingly, the input signal can be reconstructed as $\mathbf{x} = \sum_i \hat{x}_i \mathbf{u}_i$, which corresponds to the inverse graph Fourier transform.

The graph convolution of the input signal $\mathbf{x}$ with a filter $g \in \mathbb{R}^n$ is defined in Equation 2.7a, which can be simplified to Equation 2.7b, where $\mathbf{g}_\theta = \text{diag}(\mathbf{U}^T \mathbf{g})$ represents a filter. All spectral-based ConvGNNs follow this definition, but the key difference lies in the choice of the filter $\mathbf{g}_\theta$.

$$\mathbf{x} *_G \mathbf{g} = \mathcal{F}^{-1}(\mathcal{F}(\mathbf{x}) \odot \mathcal{F}(\mathbf{g})) = \mathbf{U}(\mathbf{U}^\top \mathbf{x} \odot \mathbf{U}^\top \mathbf{g}) \quad (2.7a)$$

$$\mathbf{x} *_G \mathbf{g}_\theta = \mathbf{U} \mathbf{g}_\theta \mathbf{U}^\top \mathbf{x} \quad (2.7b)$$

As a result of the eigendecomposition of the Laplacian matrix, ConvGNNs encounter three limitations [20]. First, even a minor perturbation to a graph results in the change of the eigenbasis, thus, a recomputation of the eigenbasis is needed. Second, the learned filters are domain dependent, meaning that the filters cannot be applied to a graph with a different structure. Third, eigendecomposition is computationally expensive, with a complexity of $\mathcal{O}(n^3)$.

Graph Convolutional Networks (GCN) proposed by [21] reduced the complexity to $\mathcal{O}(m)$ by making several approximations and simplifications. In fact, GCN introduces a first-order approximation of ChebNet, another spectral-based ConvGNNs. Additionally, GCN proposed a novel layer-wise propagation rule (Equation 2.8).

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} \right) \quad (2.8)$$

In the above formulation, $\tilde{A} = A + I_N$ is the adjacency matrix of the undirected graph $\mathcal{G}$ with added self-connections, indicating that the output of a node in a hidden layer depends on itself and its neighbors. I_N is the identity matrix, $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$ and $W^{(l)}$ is a layer-specific trainable weight matrix, $\sigma(\cdot)$ corresponds to an activation function, for example, a $\text{ReLU}(\cdot)$, and $H^{(l)} \in \mathbb{R}^{N \times D}$ corresponds to the matrix of activations for the l^{th} layer, thus $H^{(0)} = X$. GCN bridged the gap between the spectral-based approaches and the spatial-based approaches [20].

GCN outperformed, by a significant margin, several other models in the task of semi-supervised node-level classification [21]. However, Kipf and Welling pointed out some limitations. They assume equal importance of self-connections vs. edges to neighboring nodes, which may not hold true in all cases, furthermore, they only considered undirected graphs, which restricts the model applicability to only some scenarios.

2.5.2 Link prediction on GNNs

Liu et al. propose the use of GCNs to predict protein-protein interactions, arguing that traditional prediction methods predominantly rely on amino acid sequence data, thereby neglecting the valuable structural information embedded within PPI networks [22]. Their results emphasize this assertion by demonstrating that incorporating graph-structured information with amino acid sequence information improves prediction performance compared to solely rely on sequence information.

The proposed method is divided into two distinct phases: the **representation phase** and the **prediction phase**.

- The representation phase combines information from both the protein sequence and the graph structure, each represented by a corresponding matrix, which are then combined to form a final representation matrix. Each row of the final matrix corresponds to the feature representation of a unique protein. This phase is composed of two stages:

- *Encode Protein Sequence.* The encoding process transforms the linear sequence of amino acids, which is the primary structure of a protein, into a sparse vector consisting of natural numbers. Since there are 20 unique amino acids, the authors use one-hot encoding, representing each amino acid by a natural number. Then an identity matrix is constructed with shape 20×20 , where each line corresponds to the feature vector of a unique amino acid. With the help of this matrix, the sparse vectors of the proteins can then be reconstructed to the original dense vector;
- *Capture Graph Structure Information.* To capture the information about the protein position in the PPI network, spectral-based GCNs are used. The GCNs receive both the graph structure information, in the form of the adjacency matrix, and the node feature information, with the proteins already encoded, as inputs. In practice, each node's hidden representation is composed of its original feature information and first-order neighbor's feature information, computed through:

$$X_1^{N*f} = \sigma(\tilde{D}^{-1} \tilde{A} X_0^{N*N} W_0) \quad (2.9)$$

Here, $\tilde{A} = A + I_N$ corresponds to the adjacency matrix of the PPI networks graph with added self-connections. D is the degree matrix with $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$, W_0 is the trainable weight matrix in the first layer, and $\sigma(\cdot)$ is an activation function, for example, $\text{ReLU}(\cdot)$. X_0^{N*N} is the original feature matrix, N is the number of proteins in the graph. X_1^{N*f} is the output feature matrix, and f is the feature length of each node after GCNs operation.

- In the prediction phase, the final representation matrix is used as input to a Deep Neural Network (DNN) architecture to make predictions of whether or not there exists an interaction between the proteins. The prediction phase consists of the following components:
 - *Dual DNN Modules.* Two separate DNN modules are employed, each composed by a fully connected layer followed by a normalization layer, and then a dropout layer, this structure is repeated four times within each DNN module;
 - *Joint Module.* The outputs of the two DNN modules are combined and fed into a joint module, which consists of 2 fully connected layers followed by a normalization layer;

- *Output Layer.* The output of the joint module is processed through a fully connected layer followed by a normalization layer, and, finally, through a softmax activation layer to predict the interaction probability between proteins.

This novel representation and prediction method successfully combines amino acid sequence information with graph structure information. By incorporating graph-based representations, the method captures the positional information of proteins within PPI networks, which significantly enhances the predictive performance. The results in [22] reveal that the incorporation of network information into the model enables this approach to outperform state-of-the-art sequence-based methods. Moreover, the results also reveal that this approach not only outperforms the state-of-the-art sequence-based methods but it also is more efficient.

Zhao et al. [23] introduce SemiGNN-PPI, a self-ensembling multi-graph neural network designed to predict protein-protein interaction networks. This model operates in a multi-label scenario, where each edge is associated with multiple labels representing different types of interactions (edges) between proteins. By simultaneously modeling proteins correlations and label dependencies, SemiGNN-PPI offers a significant advantage over sequence-only methods as [22] by leveraging the graph-structured nature of PPI networks.

By integrating a Mean Teacher strategy with the graph neural network, the proposed model is able to handle unlabeled data. The goal of the model is to learn a function that learns the labels of an edge $f : e_{ij} \rightarrow \hat{y}_{ij}$. The SemiGNN-PPI framework is composed of two main components:

- *Multi-graph encoding module.* This module encodes both protein relationships and label dependencies through two sub-modules:
 - *Protein Graph Encoding (PGE) network.* Uses a GNN architecture to learn high-level compact representations of the proteins, $H \in \mathbb{R}^{|P| \times d}$, where d is the dimensionality of protein features and $h_p = H[p, :]$ is the latent representation of node p , based on the graph structure and the sequence-based protein. The GNN follows a recursive neighborhood aggregation scheme to iteratively update the representation of each node by aggregating and transforming the representation of its neighboring nodes;
 - *Label Graph Encoding (LGE) network.* Uses a GCN to learn label dependencies through inter-dependent classifiers, W , which can then be directly applied to the protein representations learned in PGE to predict multi-type PPI. Afterwards, by applying the learned classifiers W from label graph encoding (LGE) to the learned representations from protein graph encoding (PGE) for the PPI e_{ij} , the predicted labels can be calculated by $\hat{y}_{ij} = W(h_{p_i} \cdot h_{p_j})$. It is employed a traditional multi-label classification loss function to update the whole network $\mathcal{L}_{sup} = \sum_{c=1}^t (y^c \log(\sigma(\hat{y}^c)) + (1 - y^c) \log(1 - \sigma(\hat{y}^c)))$, where σ is the sigmoid function.

- *Mean Teacher Training Strategy.* This component enables the model to exploit unlabeled data by leveraging a teacher-student framework. The teacher model is updated iteratively using an exponential moving average (EMA) of the student model’s weights by leveraging the momentum updating mechanism (Equation 2.10).

$$\theta'_k = m\theta'_{k-1} + (1 - m)\theta_k \quad (2.10)$$

Here θ' and θ are the teacher model weights and student model weights, respectively, and m the momentum parameter. To enhance robustness, the model applies two graph augmentation methods at the node and edge levels, to improve the robustness against connectivity variations and attribute missing, respectively.

By employing the teacher-student strategy, the SemiGNN-PPI effectively enforces multiple graph consistency constraints at both the node and edge levels, which are utilized to enhance knowledge distillation for self-ensemble learning. The overall approach achieve efficient and generalizable multi-type PPI prediction.

2.5.3 Adversarial Training on GNNs

Adversarial Training is a dynamic regularization technique that can resist worst-case perturbations on input features and consequently improve the robustness and generalization of the model it is applied to. However, directly employing AT on graph neural networks is insufficient, since it treats examples as independent of one another failing to account for the influence of connected examples.

To address this limitation, Feng et al. [24] proposed Graph Adversarial Training, a dynamic regularization technique that accounts for the graph structure. GraphAT incorporates the influence of connected examples into account when learning to construct and resist perturbations, enhancing the model’s robustness against perturbations whilst improving generalization.

The central concept of GraphAT is to approximate as much as possible the worst case scenario by generating adversarial perturbations that target smoothness constraint of graphs, a property where closely connected nodes are encouraged to have similar predictions. Unlike conventional adversarial training, which considers examples in isolation, GraphAT maximizes the divergence between the prediction of a target node and its connected nodes when generating adversarial perturbations [24]. This approach exploits the interdependence between nodes, a fundamental aspect of graph-based learning.

These adversarial perturbations attack the graph smoothness constraint as much as possible. Then, GraphAT updates the model parameters by minimizing a graph adversarial regularizer, which reduces the divergence between the perturbed target example and its connected neighbors. This regularization encourages smoother predictions across the graph, reinforcing generalization. To compute adversarial

perturbations efficiently, a linear approximation method based on backpropagation is used.

GraphAT can be understood as a minimax game, where the model alternates between generating adversarial perturbations (maximization) and updating the model parameters accordingly (minimization). To establish a clear understanding of the underlying formulation of GraphAT, we first define some key components:

- r_i^g represents the graph adversarial perturbation which is applied to the input feature of the clean example i to construct the graph adversarial example. An adversarial example is an input created by applying small, deliberately crafted worst-case perturbations to examples from the dataset, such that the new example causes the model to produce an incorrect answer with high confidence;
- Θ denotes the parameters to be learned by the model;
- d is a non-negative function that measures the divergence between two predictions.

With these definitions, the GraphAT framework can be formalized as follows:

- *Minimization.* GraphAT minimizes the objective function of the graph neural network with an additional regularizer over adversarial examples. This graph adversarial regularizer promotes smoothness by encouraging predictions of adversarial examples to align with those of connected nodes. This results in a more robust model against perturbations that propagate through the graph.

$$\Gamma_{GAT} = \Gamma + \beta \sum_{i=1}^N \sum_{j \in \mathcal{N}_i} d(f(x_i + r_i^g, G|\Theta), f(x_j, G|\Theta)) \quad (2.11)$$

In Equation 2.11, Γ_{GAT} is the training objective function composed by the objective function, Γ , of the origin graph-based learning model, and by the graph adversarial regularizer, $\sum_{i=1}^N \sum_{j \in \mathcal{N}_i} d(f(x_i + r_i^g, G|\Theta), f(x_j, G|\Theta))$.

- *Maximization.* GraphAT generates graph adversarial perturbations, r_i^g , from clean examples, with the intention to break the smoothness between connected nodes to the maximum extent. Those perturbations will then be added to the input of clean examples, generating, thus, adversarial examples.

$$r_i^g = \underset{r_i}{\operatorname{argmax}}_{||r_i|| \leq \epsilon} \sum_{j \in \mathcal{N}_i} d(f(x_i + r_i, G|\hat{\Theta}), f(x_j, G|\hat{\Theta})) \quad (2.12)$$

The graph adversarial perturbation is determined by maximizing the graph adversarial regularizer under the current values of the model parameters. In other words, the perturbation represents the direction of change in the input features that most effectively disrupts the graph adversarial regularizer, simulating the worst-case perturbations propagated from neighbor nodes. A hyperparameter ϵ controls the magnitude of perturbations, which is typically set to small values to ensure feature distribution of adversarial examples remains similar to that of the original examples.

The idea is to prevent perturbations from propagate through node connections, the proposed method encourages the smoothness of predictions, a property that indicates a good generalization of the model.

GraphAT is a more advanced regularization than traditional graph-based regularizations for two reasons. Firstly, the regularization is dynamic since the adversarial examples are adaptively generated according to the current parameters and predictions of the model, whereas the traditional graph-based regularizations are static. Secondly, to some extent GraphAT augments the training data, since the generated adversarial examples are not present in the original training data, which benefits generalization.

The proposed method is a generic solution that can be seamlessly applied to train most graph neural networks. In [24], the method was applied to GCN for a node classification task. The results demonstrated that training GCN with the GraphAT significantly improves the robustness and generalization, highlighting the importance of adversarial training that considers graph structure.

However, GraphAT is primarily designed for perturbations on node input features and for node classification tasks. Its applicability to other tasks, such as node regression and link prediction, remains unexplored. As the authors pointed out, this presents an opportunity for future research to extend GraphAT to a broader range of graph-related problems.

3

Related Work

Contents

3.1 Generative Adversarial Imputation Networks	19
3.2 ProtoGain	20
3.3 Domain-Adversarial Neural Networks	21

This section reviews the related work relevant to the proposed model, organized into two categories: the Generative Adversarial Imputation Networks approaches [6, 15], and the Domain-Adversarial Neural Network [25].

3.1 Generative Adversarial Imputation Networks

As briefly described in Section 2.3, GAIN extend the concept of GAN for imputation of missing values. Similar to GANs, GAIN consists of two main components modeled as fully connected neural networks: the generator G and the discriminator D .

GAIN operates on an incomplete dataset, X , referred to as the original matrix. To facilitate the imputation process, two additional matrices are derived from X , the data matrix $\tilde{X}$ and the mask matrix M . The data matrix is constructed by replacing all the missing entries in X with zeros while retaining

the observed values unchanged. The mask matrix, is a binary matrix where each entry is 1 if the corresponding value in X was observed and 0 otherwise. Before constructing $\tilde{X}$, the features of X are normalized to ensure consistency across different scales. Missing entries in X are further augmented with random noise sampled from a uniform distribution, represented by the noise matrix Z , where the entries corresponding to missing values are replaced with random noise.

The generator G takes $\tilde{X}$, M , and Z as inputs and outputs an imputation matrix $\bar{X}$, which contains synthetic imputed values for both observed and missing entries. The final matrix $\hat{X}$ is formed by combining the observed values of X with the synthetic values generated by G (Equation 3.1).

$$\begin{aligned}\bar{X} &= G(\tilde{X}, M, (1 - M) \odot Z) \\ \hat{X} &= M \odot \tilde{X} + (1 - M) \odot \bar{X}\end{aligned}\tag{3.1}$$

Here, $\odot$ denotes the element-wise multiplication. Unlike standard GANs, where the discriminator distinguishes between real and generated samples, the discriminator D in GAIN predicts whether each entry in $\hat{X}$ was observed or imputed. To enhance the discriminator's ability to differentiate observed and imputed values, a hint matrix, H , is introduced. The hint matrix provides partial information through probabilities about which entries are observed or missing according to a hint rate, helping to guide the discriminator. D receives both $\hat{X}$ and H as inputs and outputs an estimated mask matrix, assigning probabilities to each entry of $\hat{X}$ being observed or imputed.

GAIN is trained via a minimax game between the generator and the discriminator. The generator G is optimized to minimize the discrepancy between its imputations and the observed data, effectively fooling the discriminator into treating all entries in $\hat{X}$ as observed. The discriminator D , on the other hand, is optimized to accurately reconstruct the mask matrix M , leveraging the hint matrix H to improve its predictions. The adversarial training process alternates between updating G and D ensuring that G learns to generate realistic imputations while D improves its ability to distinguish observed from imputed entries.

3.2 ProtoGain

While GAIN [5] was tested with synthetic values, work in our group [6] and its implementation counterpart ProtoGain¹, extended its application to a real-life scenario in proteomics data. ProtoGain preserved the core architecture of GAIN, with both the generator G and the discriminator D implemented as fully connected neural networks composed by layers of dimensions $2m$, m , and m , respectively, where m corresponds to the number of features (proteins) in the dataset. The activation functions used were the *tanh*, *tanh*, and *sigmoid*, respectively, and the features values were scaled to the interval $[0, 1]$ using a

¹<https://github.com/QuantitativeBiology/GenerativeProteomics>

Min-Max normalization prior to training. The training process follows the same procedure described in Section 3.1.

A detailed study was conducted in [6] analyzing the correlation between missing values and model performance, benchmarking its performance against imputation models, namely missForest, and exploring hyperparameter optimization. Four key findings were found. First, the optimization process revealed that batch size had minimal impact on performance. Second, a low number of iterations helped avoiding spikes in the loss while reducing computational time. Third, smaller values for the hint rate h favored generalization. Lastly, larger values for α , which controls the losses weights, led to lower Root Mean Square Error (RMSE). A higher α strengthens the reconstruction term of the loss, which in turn encourages the generator to reproduce more closely the observed elements, favoring replication.

Performance across different intervals of missingness showed that GAIN outperformed missForest, regardless of the number of trees used. However, the imputations values generated by GAIN tended to be systematically lower and less variable. This suggests that GAIN might be generating conservative estimates, reducing extreme imputed values but potentially underestimating true variability. Nevertheless, GAIN still yielded more variance in its imputations than missForest. This is an advantage, as most missing data in the study followed a MNAR pattern. In terms of computational time, GAIN reaffirmed itself as a solid option. It consistently achieved competitive RMSE compared to missForest while requiring substantially less processing time, reinforcing its suitability for large-scale proteomics datasets.

Throughout the development of the proposed model in this work, ProtoGain was consistently used to benchmark our results. As such, it is important to note that at the time of ProtoGain’s development, data availability was limited. Consequently, its implementation did not include a strict separation between training and testing data, with its model evaluation being performed on a subset of the training set. This overlap between train and test partitions may introduce a positive bias in its results, especially when dealing with smaller datasets.

3.3 Domain-Adversarial Neural Networks

Among the paradigms introduced in Section 2.4, Domain-Adversarial Neural Networks constitute a key example of adversarial-based deep transfer learning. Proposed by Ganin et al. [25], DANN introduced a novel approach to representation learning for domain adaptation (DA). Domain adaptation is a well-known challenge in transfer learning, occurring when a model trained on a source domain $\mathcal{D}_S$ must be applied to a target domain $\mathcal{D}_T$ that follows a similar but distinct distribution. DANN addresses this issue by embedding domain adaptation directly into the representation learning process, enabling the extraction of features that generalize across domains.

The DANN framework is grounded in the domain adaptation theory originally formalized by Ben-

David et al. [26], particularly in the notion of $\mathcal{H}$ -divergence, which quantifies the discrepancy between two distributions with respect to a hypothesis class $\mathcal{H}$. A low $\mathcal{H}$ -divergence implies that a representation is unable to discriminate whether a sample originates from $\mathcal{D}_S$ or $\mathcal{D}_T$. According to this theory, successful transfer requires features that simultaneously minimize the classification error on the source domain and the $\mathcal{H}$ -divergence between source and target domains. DANN capitalizes on this idea by training its feature extractor to minimize the label prediction loss on the source domain while maximizing the error of a domain classifier that seeks to discriminate between $\mathcal{D}_S$ and $\mathcal{D}_T$.

The DANN architecture can be extended to almost any feed-forward network with a few standard layers and a non-standard *Gradient Reversal Layer (GRL)*. It consists of three modules:

- a **feature extractor** $G_f(\cdot, \theta_f)$, which maps an input $\mathbf{x} \in \mathbb{R}^m$ into a D -dimensional latent representation;
- a **label predictor** $G_y(\cdot, \theta_y) : \mathbb{R}^D \rightarrow [0, 1]^L$, trained to classify source samples into L categories;
- a **domain classifier** $G_d(\cdot, \theta_d) : \mathbb{R}^D \rightarrow [0, 1]$, trained to distinguish between source and target domains, where samples from the source domain are labeled 0 and those from the target domain are labeled 1.

The label predictor minimizes a classification loss $\mathcal{L}_y$, such as a logistic regression loss, over the labeled source data. In parallel, the domain classifier minimizes a binary cross-entropy loss $\mathcal{L}_d$ that penalizes incorrect domain predictions, while the feature extractor is trained adversarially to minimize the $\mathcal{L}_y$ while maximizing the $\mathcal{L}_d$. This adversarial interplay between G_f and G_d encourages the feature extractor to learn representations that are invariant to domain differences, fooling the domain classifier, while remaining discriminative for the main predictive task.

This adversarial training is implemented efficiently through the gradient reversal layer, which enables the backpropagation of reversed gradients without modifying the network architecture. The gradient reversal layer acts as the identity function during the forward pass (Equation 3.2a), but multiplies the incoming gradients by -1 during backpropagation before passing them to the feature extractor (Equation 3.2b).

$$\mathcal{R}(\mathbf{x}) = \mathbf{x} \tag{3.2a}$$

$$\frac{\partial \mathcal{R}(\mathbf{x})}{\partial \mathbf{x}} = -\mathbf{I} \tag{3.2b}$$

This layer is inserted between the G_f and G_d , and its sole function is to reverse the gradient between the domain classifier and the feature extractor, ensuring that the feature extractor is updated to maximize

the domain loss while the domain classifier minimizes it. As such, the GRL has no learnable parameters associated with it.

Formally, let $\mathcal{L}_y(G_y(G_f(\mathbf{x})), y)$ denote the label prediction loss for the source sample $(\mathbf{x}, y)$, and $\mathcal{L}_d(G_d(G_f(\mathbf{x})), d)$ the domain classification loss, where $d \in \{0, 1\}$ indicates source or target domain. The overall objective is formulated as:

$$\begin{aligned} E(\theta_f, \theta_y, \theta_d) = & \frac{1}{n} \sum_{i=1}^n \mathcal{L}_y(G_y(G_f(\mathbf{x}_i; \theta_f); \theta_y), y_i) \\ & - \lambda \left(\frac{1}{n} \sum_{i=1}^n \mathcal{L}_d(G_d(\mathcal{R}(G_f(\mathbf{x}_i; \theta_f))); \theta_d), d_i \right) \\ & + \frac{1}{n'} \sum_{i=n+1}^{n'} \mathcal{L}_d(G_d(\mathcal{R}(G_f(\mathbf{x}_i; \theta_f))); \theta_d), d_i \Big) \end{aligned} \quad (3.3)$$

In Equation 3.3 the domain adaptation parameter λ balances the trade-off between minimizing the classification loss and reducing $\mathcal{H}$ -divergence. Training seeks a saddle point, as shown in Equation 3.4. Here, n denotes the number of source samples, n' the number of target samples, and $N = n + n'$ the total number of samples.

$$(\hat{\theta}_f, \hat{\theta}_y) = \operatorname{argmin}_{\theta_f, \theta_y} E(\theta_f, \theta_y, \hat{\theta}_d) \quad (3.4a)$$

$$\hat{\theta}_d = \operatorname{argmax}_{\theta_d} E(\hat{\theta}_f, \hat{\theta}_y, \theta_d) \quad (3.4b)$$

This optimization can be performed efficiently using stochastic gradient descent, where updates for θ_f are reversed in sign for the domain loss by the GRL.

This training ensures that at convergence, the feature extractor has learned features that are both domain-discriminative for the main learning task on the source domain and domain-invariant with respect to the shift between the domains.

Although originally proposed for classification tasks, the DANN framework is both task and architecture agnostic. The only requirement is a shared feature representation that can be trained to become domain invariant. Consequently, the framework only requires a feature extractor and a gradient reversal layer to enable the learning of domain-invariant features while preserving task-specific discriminative power.

4

The GAIN-DANN Model

Contents

4.1 Model Design	26
4.2 Training	31
4.3 Evaluation	37
4.4 Inference	37
4.5 Hyperparameters Optimization	38
4.6 Imputation with a pretrained model	39
4.7 Implementation and Usage Details	40

Proteomics datasets are often characterized by substantial missingness and domain-specific variability introduced by differences in the experimental design, instrumentation and sample preparation. While ProtoGain has demonstrated strong performance when trained and applied within individual datasets (Section 3.2), we hypothesized that its effectiveness can be further enhanced by leveraging information learned from other related proteomics datasets. In particular, knowledge learned from larger and more complete datasets can improve imputation in smaller and/or noisier datasets. However, effectively transferring such knowledge is non-trivial due to distributional shifts and differences in dimensionality between datasets. To address this challenge, we propose a hybrid model, GAIN-DANN, that integrates

the representation learning capabilities of the Domain-Adversarial Neural Networks with the imputation mechanism of GAIN. The proposed model is designed to promote domain-invariant feature learning while maintaining an accurate reconstruction of the missing protein intensities, thereby enabling cross-project transfer for improved imputation.

The intuition behind the proposed model stems from the premise that imputation quality can be improved by transferring relevant knowledge across proteomics datasets. Standard imputation models such as GAIN are typically trained independently on each dataset, learning local distributions. By leveraging knowledge from other related datasets, it becomes possible to capture information that may be missing or underrepresented in a single dataset, thus producing imputations that are more consistent with true biological variation. The DANN framework was developed to mitigate domain shifts by encouraging the emergence of latent representations that are both discriminative for the main learning task and invariant with respect to the domain. By considering this principle into the imputation process, our model leverages shared biological information across related datasets while minimizing dataset-specific information. This design allows the model to benefit from representations learned on larger or more complete datasets, ultimately enhancing imputation quality in small, highly incomplete target domains.

The canonical DANN model was originally designed for classification tasks, as such the task-specific module in the original DANN is a label predictor. In our adaptation, we replaced the label predictor by a GAIN module, shifting the focus from classification to missing values imputation. This allows the model to learn latent representations that remain aligned across domains while learning to impute the missing values.

In the subsequent sections, from Section 4.1.1 to 4.7, we provide a comprehensive description of the proposed model, covering its input details, architecture, training and evaluation procedures, the inference process, the hyperparameter optimization and implementation details.

4.1 Model Design

This section describes the design of the proposed model, detailing its main modules and overall structure. The design was guided by the need to effectively address missing data in proteomics while accounting for domain shifts, resulting in an architecture that combines generative imputation and adversarial domain adaptation mechanisms. Section 4.1.1 details both the input representation and the preprocessing steps to prepare the data for model training. Section 4.1.2 presents the model architecture.

4.1.1 Input

Some of the preprocessing steps differ slightly between training and imputation tasks, due to the inherent nature of such tasks. For instance, while imputation tasks operate on incomplete data, training an

imputation model requires introducing artificial missingness so that imputations can be evaluated against known ground-truth values. Below are described the sequence of preprocessing steps. For each step it is clearly stated whether it differs between the training and imputation stages.

Dataset format. The protein abundance matrix is used directly, with samples represented as rows and proteins represented as columns. For training purposes, our model expects that each sample is associated with a domain label. This information is crucial for the domain-adversarial component of our model. In the present implementation, these domain labels must be provided in a dedicated column explicitly named "Domain". For example, for the HeLa dataset used during development, this column specifies for each sample the project it belongs to, described in more detail in Section 5.1.1. Once the model is trained, domain labels are no longer required, and only the protein abundance matrix is needed. Thus, the dataset must be in a CSV file formatted accordingly. When used for training the protein abundance matrix must be accompanied by a column "Domain", when used for imputation only the protein abundance matrix is required. All subsequent data operations: dataset loading, normalization, and missingness induction (if applicable) are handled by a `Data` class.

Log-transformed data. Although training our model from scratch does not strictly require log-transformed data, it is generally preferable to do so when dealing with protein abundance values. Raw protein intensities typically span several orders of magnitude and exhibit a highly skewed distribution. This wide range can impair the performance of the model by introducing extreme values. As such, we recommend that when possible log-transformed data should be preferred to non-transformed data as input for our model.

Normalization. After that, the data is standardized using a *Standard Scaler*, such that each protein has a mean of zero and a unit variance. The missing values are not considered in the normalization step. This normalization step is applied both during training and when imputing on unseen datasets.

Induced missingness for training. To train the model, additional missing values are artificially introduced in the data to simulate incomplete data. For instance, while imputation tasks operate on incomplete data, training an imputation model requires introducing missingness so that imputations can then be evaluated against known ground-truth values. By default, the missing rate is set to 0.1 (10%), although this setting can be adjusted by the user through the parameter file (`miss_rate`). This step is specific to the training phase.

4.1.2 Architecture

The design of our model is guided by two main objectives. The imputation of missing protein abundances while ensuring that the imputations are agnostic to batch effects. To clarify the design choices, we motivate the need for each module while simultaneously explaining how they jointly address the imputation task.

A central motivation of our work is to exploit the knowledge learned from one proteomics dataset to improve the imputation performance in other datasets. This is motivated by two considerations. First, it is reasonable to assume that a protein may be undetected in one dataset while being present in another, due to factors such as variations in experimental setups, differences in instrumentation sensitivity or the stochastic nature of mass-spectrometry. Thus, a model trained on the latter should, in principle, be able to leverage the knowledge learned to impute its abundance in the former. Second, most machine learning models require a substantial quantity of data to generalize well. This implies that learning a model on a smaller dataset for imputation has additional challenges. Thus, leveraging knowledge learned from larger datasets arises as a promising strategy. Both of these motivations place our work within the paradigm of transfer learning, whose goal simply put is to transfer knowledge from one domain to another.

However, proteomics data are usually affected by batch effects, inherent to technological limitations in proteins measurements, which do not reflect true biological variations. As such, these batch effects can severely distort the representations learned by the model and hinder its transfer. Addressing batch effects requires, therefore, tackling these domain shifts, which can be achieved through domain adaptation mechanisms. The DANN architecture (see Section 3.3), has shown promising results in mitigate these domain shifts [25]. Its success relies in an adversarial training scheme: while the encoder tries to learn domain-invariant and discriminative representations, a domain classifier is simultaneously trying to predict the domain label. Therefore, this interplay between the encoder and the domain classifier, ensures that the learned representations by the encoder are not dominated by dataset-specific biases but instead capture biologically meaning variation, an example can be seen in Figure 5.5. This is of extremely importance for our approach, as it provides a mechanism that can mitigate batch effects. Thus, this justifies the inclusion of the encoder and domain classifier modules in our architecture.

Once such domain-invariant representation has been learned, the central task of imputation must be addressed. To this end, we incorporate the GAIN module, originally introduced in ProtoGain, which is specifically designed to learn to impute missing values. However, these imputations learned by GAIN are in the latent space learned by the encoder, different from the original protein abundance space. As such, to have faithful imputations to the original space, we need to ultimately revert to the original protein abundance space. For this reason, we introduce a decoder responsible for mapping the imputed latent representations back into the original protein space. This guarantees that the output of our model is expressed in the same space as the input data, ensuring that the imputed abundances can be seamlessly used in downstream proteomics analysis.

Encoder

Unlike classical auto-encoders, which typically compress the data into a lower-dimensional latent space, the encoder in our architecture is designed in the context of imputation. The encoder works

primarily as a feature extractor that maps the proteins to a latent representation $\mathbf{z} \in \mathbb{R}^D$, which, in theory, should obfuscate domain-specific information. We enforce $D = m$, with m corresponding to the input dimension (number of proteins). This is necessary since our main goal is imputation, which ultimately requires a return to the original input dimension.

Formally, the encoder can be expressed as a feedforward neural network parametrized by θ_f (Equation 4.1).

$$\mathbf{z} = G_f(\mathbf{x}; \theta_f), G_f : \mathbb{R}^m \rightarrow \mathbb{R}^D \quad (4.1)$$

The encoder receives the protein feature vector $\mathbf{x} \in \mathbb{R}^m$ as input and maps it into a hidden representation in $\mathbb{R}^h$, where $h = \frac{1}{2}m$, through a fully connected layer followed by batch normalization, a ReLU activation, and dropout. Subsequently, the encoder contains a configurable number of hidden blocks, each consisting of a fully connected layer ($\mathbb{R}^h \rightarrow \mathbb{R}^h$), batch normalization, ReLU activation, and dropout. The number of such blocks is treated as a tunable hyperparameter. Finally, a fully connected output layer projects the resulting representation into the latent space $\mathbb{R}^D$. The dropout probability is the same across all the dropout layers of both the encoder and the decoder. This hyperparameter can be tuned by the user in the parameters file.

The systematic use of batch normalization, followed by a ReLU activation and a dropout layer is intended to help the encoder produce a more stable, domain-invariant representation of the data. The batch normalization improves training stability while accelerating convergence. The ReLU activation introduces non-linearity allowing the encoder to capture more complex, non-linear dependencies among the protein features. Finally, the dropout layer provides stochastic regularization, reducing the risk of overfitting while encouraging more generalizable representations.

Gradient Reversal Layer

The Gradient Reversal Layer ensures the adversarial training nature of our model's training. As motivated in this section, our goal is to enforce the encoder to learn domain-invariant features while still preserving discriminative information. The GRL helps this interaction by pairing the encoder with the domain classifier in an adversarial manner.

Formally, the GRL has no parameters associated with it. Since in the forward pass, it behaves as the identity function, passing the latent representation $\mathbf{z}$ unchanged to the domain classifier (Equation 4.2a), and in the backward pass it multiplies the gradient by a negative scalar λ_p before passing it to the encoder (Equation 4.2b), where $\mathbf{I}$ denotes the identity matrix.

$$\mathcal{R}(\mathbf{z}) = \mathbf{z} \quad (4.2a)$$

$$\frac{d\mathcal{R}}{d\mathbf{z}} = -\lambda_p \mathbf{I} \quad (4.2b)$$

This simple, yet crucial operation ensures that while the domain classifier attempts to minimize the domain classification loss, the encoder receives the reversed gradient, and is thus encouraged to maximize the domain classification loss. This results in features representations that obfuscate domain bias features, thus mitigating batch effects.

In practice, the GRL offers a mechanism to implement a min-max optimization within a standard backpropagation without modifying the optimizer. This formulation was originally introduced in the DANN architecture [25].

Domain Classifier

The domain classifier receives the latent representation $\mathbf{z}$ through the GRL, and attempts to predict its domain label $\hat{y}$. Formally, the domain classifier $G_y(\cdot)$, is defined as a function that maps the latent representation to a domain prediction (Equation 4.3).

$$\hat{y} = G_y(\mathcal{R}(\mathbf{z}); \theta_y) \quad (4.3)$$

The output $\hat{y}$ lies in $\in \{y_1, \dots, y_K\}$ where K is the number of domains. The domain classifier is implemented as a two-layer feedforward neural network with a ReLU activation in between. To avoid giving the domain classifier excessive discriminative power, which would hinder the encoder's ability to learn domain-invariant features, we set the hidden dimension to 128.

It is important to note that our domain classifier differs from the canonical DANN. In the original DANN, the domain classifier is formulated for binary classification to distinguish between a source domain and a target domain. We extend the domain classifier to be a multi-class logistic regressor, enabling it to handle multiple domains.

Finally, the domain classifier is only used during training. Its sole purpose is to enforce the adversarial objective that forces the encoder to produce domain-invariant latent representations. Once training is complete, the domain classifier is discarded.

GAIN

The imputation mechanism of our architecture is based on the GAIN framework described in further detail in Section 3.1. We adopt the same generator-discriminator structure, where the generator learns to impute missing values and the discriminator seeks to distinguish between observed and imputed entries. Both the formulation and the training of GAIN, including its use of mask and hint matrices, are

presented in Section 3.1.

As suggested earlier, in our model the imputations are not performed directly in the protein abundance space, but rather in the latent space $\mathbf{z}$ learned by the encoder. In this space, the imputations are produced by the generator network $G_{\text{GAIN}}(\cdot)$ (Equation 4.4).

$$\hat{\mathbf{z}} = G_{\text{GAIN}}(\mathbf{z}; \theta_f) \quad (4.4)$$

It is worth emphasizing that the imputation component of our model is modular. Although, we employ GAIN due to its success in proteomics settings, this module can be replaced by any other imputation network, without altering the rest of the architecture.

Decoder

The decoder is responsible for reconstructing the original protein feature space from the latent representation after imputation. While the GAIN effectively imputes the missing values, these imputations lie in the latent space. To ensure that the imputations are faithful to original protein abundance space, it is necessary to map the imputed latent representations back to real protein space.

Formally, the decoder G_d is a feedforward neural network parametrized by θ_d , that receives the latent representation with missing values imputed by the GAIN, $\hat{\mathbf{z}}$, and maps it to the reconstructed protein abundances, $\hat{\mathbf{x}} \in \mathbb{R}^m$ (Equation 4.5).

$$\hat{\mathbf{x}} = G_d(\hat{\mathbf{z}}; \theta_d) \quad (4.5)$$

The decoder mirrors the architecture of the encoder, but its output layer projects the imputed latent representation back to the input dimensionality, m .

4.2 Training

The training of our model is designed to achieve two objectives: learning domain-invariant representations to allow knowledge transfer, and accurately imputing missing protein abundances. This subsection details the procedure by which the model parameters are optimized, the role of missingness masks, and the interaction between the modules of our model. We also clarify how training occurs in both the latent and original feature spaces, and how losses are computed for each component to guide the overall training.

The role of missingness masks. Training an imputation model requires access to ground-truth values, against which imputations can be evaluated. As a result, additional artificial missingness is induced in the training data. Let $X \in \mathbb{R}^{n \times m}$ denote the original dataset with n samples and m proteins, which may or may not have missingness. Introducing artificial missingness produces $X_{\text{miss}} \in \mathbb{R}^{n \times m}$

which is used to train the model.

To correctly handle missingness, it is crucial to have two types of masks:

- **True missingness mask** M_{true} depicts which entries were genuinely missing, represented as 0, in the original dataset X , while the observed entries are represented as 1. This mask is used in the evaluation of the reconstruction of the original protein space.
- **Induced missingness mask** M_{ind} indicates which entries were artificially deleted for training. This mask is required by the GAIN module, which must learn to impute artificially missing values while preserving the observed ones.

The necessity of these two masks becomes clear when considering the multiple spaces in which learning occurs. While the encoder receives samples in the original feature space, the GAIN module operates on the latent space learned by the encoder. Then the decoder maps back to the original feature space. As such, the evaluation differs depending on the module:

- The **decoder** is evaluated against the original dataset X , using M_{true} , to assess the capacity of the decoder to model back the original feature space.
- The **GAIN** module is trained and evaluated on imputations in the latent space, using M_{ind} to specify which entries were observed and which were artificially missing.

As such, our model is essentially learning how to reconstruct the original feature space and imputation in the latent space, addressing two objectives: the mitigation of batch effects through domain adaptation and learning robust imputations through GAIN.

Training procedure. Let $X \in \mathbb{R}^{n \times m}$ denote the protein abundance matrix, and $X_{\text{miss}} \in \mathbb{R}^{n \times m}$ denote the corresponding protein abundance matrix with missingness induced. Consider also $\mathbf{x}_i$ to be a sample of the X_{miss} .

The encoder $G_f(\cdot; \theta_f)$ maps the input $\mathbf{x}$ into a D -dimensional latent representation $\mathbf{z}$ (Equation 4.6). We enforced $D = m$ to guarantee that imputations can be modeled back to the original protein space.

$$\mathbf{z} = G_f(\mathbf{x}; \theta_f), \mathbf{z} \in \mathbb{R}^D \quad (4.6)$$

To encourage domain-invariant features, the encoder tries to deceive the domain classifier, while the domain classifier aims to distinguish the domains. In other words, the encoder tries to maximize the domain classifier loss while the domain classifier tries to minimize it. This is achieved thanks to the GRL, which is inserted between both these modules. Thus, the latent representation of the data, $\mathbf{z}$, is passed to the GRL before being passed to the domain classifier. This is essential, since this gradient reversal layer behaves differently during the forward and the backward pass. On the forward pass, the GRL acts

as the identity function. But on the backpropagation, it receives the gradient from the subsequent level, the domain classifier, and changes its signal, by multiplying it by a scalar $-\lambda_p$, before passing it to the preceding layer, the encoder. This guarantees that while the domain classifier is trained to minimize its classification loss, the encoder is trained adversarially to maximize it. It is worth noting that the GRL has no parameters associated with it, and thus is not "trained" per se. The domain adaptation parameter, λ_p , is inspired by a schedule in [25], initially starting at 0 and gradually increasing to 1, following the schedule described in Equation 4.7, where p represents the training progress linearly increasing from 0 to 1.

$$\lambda_p = \frac{2}{1 + \exp(-10p)} - 1 \quad (4.7)$$

This schedule allows the domain classifier to be less sensitive to noisy signals in the early stages of training. Note that by assigning $\lambda_p = 0$, the domain adversarial component of the architecture is deactivated.

The domain classifier is a multi-class logistic regressor that maps the latent representation $\mathbf{z}$ to a predicted class label $\hat{\mathbf{y}}$ for the original input $\mathbf{x}$ (Equation 4.8). Its objective is the cross-entropy loss (Equation 4.9).

$$\hat{\mathbf{y}} = G_y(\mathbf{z}; \theta_y), G_y : \mathbb{R}^D \rightarrow \{y_1, \dots, y_K\} \quad (4.8)$$

$$\mathcal{L}_{\text{domain}}(\theta_f, \theta_y) = \lambda_p \frac{1}{n} \sum_{i=1}^n \ell_{\text{CE}}(G_y(\mathcal{R}(\mathbf{z}), \mathbf{y})) \quad (4.9)$$

Then the GAIN models a function, $G_{\text{GAIN}}(\cdot; \theta_f)$, that receives the latent vector $\mathbf{z}$ together with M_{ind} , and produces the imputed latent representation, $\hat{\mathbf{z}}$. The training of GAIN occurs in isolation from the other modules. Its parameters, are updated solely on its own objective, which minimizes the discrepancy between the imputed and observed latent entries. This ensures that GAIN learns to accurately reconstruct missing values in latent space without directly influencing the encoder or decoder during its own optimization.

Although GAIN is trained independently, it is of extremely importance that the encoder learns to represent the data in such way that the imputations are correct. To achieve this, the imputation loss is also computed in the context of the full model and used to guide the updates of the encoder. Specifically, the encoder incorporates the mean square error between the imputed latent entries and the original latent representation (Equation 4.10).

$$\mathcal{L}_{\text{imputation}}(\theta_f) = \frac{\sum_{i=1}^m \|M_{\text{ind}}^{(i)} \odot (\hat{\mathbf{z}}_i - \mathbf{z}_i)\|^2}{\sum_{i=1}^m M_{\text{ind}}^{(i)}} \quad (4.10)$$

Here, $\odot$ denotes the element-wise product. This design ensures that the encoder adapts its latent representations in a way that supports accurate imputation.

To ensure that imputations are valid in the original protein space, the decoder $G_d(\cdot; \theta_d)$ must map the latent imputations back to the original space (Equation 4.11), resulting in the reconstructed values $\hat{\mathbf{x}}$.

$$\hat{\mathbf{x}} = G_d(\hat{\mathbf{z}}; \theta_d) \quad (4.11)$$

The reconstruction is then evaluated through the root mean squared error only on the observed entries, as indicated by the M_{true} mask (Equation 4.12), where MSE corresponds to the mean square error.

$$\mathcal{L}_{\text{reconstruction}}(\theta_d, \theta_f) = \sqrt{\frac{\sum_{i=1}^m \|M_{\text{true}}^{(i)} \odot (\hat{\mathbf{x}}_i - \mathbf{x}_{\text{true}_i})\|^2}{\sum_{i=1}^m M_{\text{true}}^{(i)}}} = \sqrt{\text{MSE}(\hat{\mathbf{x}} \odot M_{\text{true}}, \mathbf{x}_{\text{true}} \odot M_{\text{true}})} \quad (4.12)$$

We can frame our problem as a minimax problem (Equation 4.13a), where the model loss $\mathcal{L}_{\text{model}}$ is a weighted sum of the imputation, reconstruction, and domain losses (Equation 4.13b) with α, β, γ as the corresponding weights for each loss and treated as hyperparameters.

$$\min_{\theta_f, \theta_d} \max_{\theta_y} \mathcal{L}_{\text{model}} \quad (4.13a)$$

$$\mathcal{L}_{\text{model}} = \alpha * \mathcal{L}_{\text{imputation}} + \beta * \mathcal{L}_{\text{reconstruction}} + \gamma * \mathcal{L}_{\text{domain}} \quad (4.13b)$$

Since imputation quality is our main focus, reflected by $\mathcal{L}_{\text{imputation}}$ and $\mathcal{L}_{\text{reconstruction}}$, we set $\beta, \alpha > \gamma$ to prioritize reconstruction over domain adaptation.

For evaluation purposes, we only looked at the losses that give insight into the task-specific objective of our model: imputation. As such, the task-specific loss is a weighted sum between the imputation capacity and the reconstruction ability of our model (Equation 4.14).

$$\mathcal{L}_{\text{task-specific}} = \alpha * \mathcal{L}_{\text{imputation}} + \beta * \mathcal{L}_{\text{reconstruction}} \quad (4.14)$$

The training procedure is detailed in the pseudo-code provided in Algorithm 4.1 and the overall architecture of the proposed model is illustrated in Figure 4.1. The GAIN component follows the generic training procedure described in Section 3.1, where the discriminator and generator are updated through the corresponding update functions implemented in the ProtoGain module.

Training details All modules and procedures are implemented in PyTorch. The model is trained following a stochastic gradient descent strategy with the *AdamW* optimizer, which decouples weight

decay from the gradient update, thereby improving generalization and training stability compared to the standard Adam optimizer. The learning rate and weight decay values were specified in the configuration file. Gradient clipping with a maximum norm of 1.0 was applied to prevent gradient explosion.

An early stopping strategy with a default patience of 10 epochs, which can be changed through the `early_stop_patience` on the parameter file, was employed to prevent overfitting. A learning rate scheduler was also applied, more specifically the *ReduceLROnPlateau* was chosen, since it allows to adapt the learning rate on the validation RMSE, which better reflects our main objective. When no improvement in the RMSE is observed for 8 consecutive epochs, the learning rate is reduced by a factor of 0.5, with a minimum learning rate of 1×10^{-6} . This scheduler was chosen because it directly reacts to stagnation in the true imputation performance metric (RMSE), rather than relying solely on the training loss.

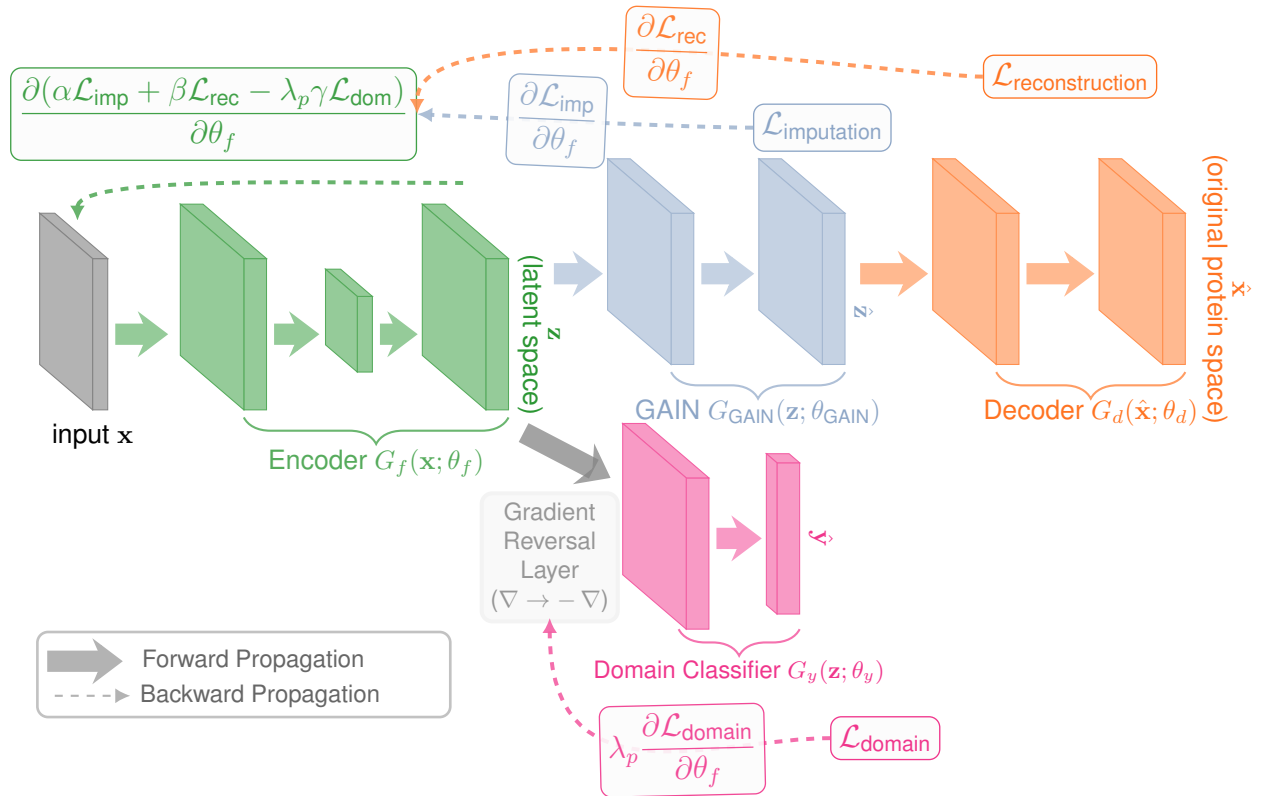


Figure 4.1: The proposed GAIN-DANN architecture includes an encoder (green), a gradient reversal layer (gray) inserted between the encoder and domain classifier (pink), a GAIN module (blue), and a decoder (orange). For clarity, loss terms are abbreviated as $\mathcal{L}_{\text{rec}}$ for the reconstruction loss, $\mathcal{L}_{\text{imp}}$ for the imputation loss, and $\mathcal{L}_{\text{dom}}$ for the domain loss.

Algorithm 4.1: Stochastic gradient descent training procedure of GAIN-DANN

Input: Protein abundance matrix $X \in \mathbb{R}^{n \times m}$;

Protein abundance matrix with induced missingness $X_{\text{ind}} \in \mathbb{R}^{n \times m}$;

Domain labels $Y \in \mathbb{R}^n$;

Learning rate η

1 **while** stopping criterion not met **do**

2 **for** $i \leftarrow 1$ to n **do**

3 $\mathbf{x}_i \leftarrow X_{\text{ind}}[i], \mathbf{x}_{\text{true}} \leftarrow X[i]$

4 $\mathbf{y}_i \leftarrow Y[i]$

5 $M_{\text{true}}[i, j] \leftarrow 1$ if $X[i, j]$ observed, else 0

6 $M_{\text{ind}}[i, j] \leftarrow 1$ if $X_{\text{ind}}[i, j]$ observed, else 0

7 # Forward Propagation

8 $\mathbf{z}_i \leftarrow G_f(\mathbf{x}_i; \theta_f)$

▷ Encoder (G_f)

9 $\hat{\mathbf{y}}_i \leftarrow G_y(\mathcal{R}(\mathbf{z}_i); \theta_y)$

▷ Domain Classifier (G_y)

10 # Update of GAIN

11 $\hat{\mathbf{z}}_i \leftarrow G_{\text{GAIN}}(\mathbf{z}_i; \theta_f)$

▷ GAIN (G_{GAIN})

12 $\hat{\mathbf{x}}_i \leftarrow G_d(\hat{\mathbf{z}}_i; \theta_d)$

▷ Decoder (G_d)

13 # Backward Propagation

14 # Compute Losses

15 $\mathcal{L}_{\text{domain}}(\theta_y, \theta_f) \leftarrow \text{CrossEntropy}(\hat{\mathbf{y}}_i, \mathbf{y}_i)$

16 $\mathcal{L}_{\text{imputation}}(\theta_f) \leftarrow \text{MSE}(\hat{\mathbf{z}}_i \odot M_{\text{ind}}^{(i)}, \mathbf{z}_i \odot M_{\text{ind}}^{(i)})$

17 $\mathcal{L}_{\text{reconstruction}}(\theta_d, \theta_f) \leftarrow \text{RMSE}(\hat{\mathbf{x}}_i \odot M_{\text{true}}^{(i)}, \mathbf{x}_{\text{true}_i} \odot M_{\text{true}}^{(i)})$

18 $\mathcal{L}_{\text{model}}(\theta_f, \theta_y, \theta_d) \leftarrow \alpha * \mathcal{L}_{\text{imputation}} + \beta * \mathcal{L}_{\text{reconstruction}} + \gamma * \mathcal{L}_{\text{domain}}$

19 # Update parameters

20 $\theta_d \leftarrow \theta_d - \eta \nabla_{\theta_d} \mathcal{L}_{\text{reconstruction}}$

21 $\theta_y \leftarrow \theta_y - \eta \lambda_p \nabla_{\theta_y} \mathcal{L}_{\text{domain}}$

22 $\theta_f \leftarrow \theta_f - \eta (\nabla_{\theta_f} (\alpha * \mathcal{L}_{\text{imputation}} + \beta * \mathcal{L}_{\text{reconstruction}}) - \lambda_p \nabla_{\theta_f} \mathcal{L}_{\text{domain}})$ ▷ - λ_p , since the

GRL inverts the gradient

23 **end**

24 **end**

4.3 Evaluation

We employed a 5-fold stratified cross-validation procedure. We opted for the stratified variant to accommodate datasets with imbalanced domain distributions, thus avoiding potential biases that could arise from unequal representation across folds.

During training, we monitored several metrics at every epoch: the GAIN loss ($\mathcal{L}_{\text{imputation}}$), the domain classifier loss ($\mathcal{L}_{\text{domain}}$), the domain classification accuracy, the overall model loss ($\mathcal{L}_{\text{model}}$), the task-specific loss ($\mathcal{L}_{\text{task-specific}}$) and the reconstruction error ($\mathcal{L}_{\text{reconstruction}}$, measured as RMSE). These metrics provide complementary insights into the learning progress. Specifically, we expect $\mathcal{L}_{\text{imputation}}$ to decrease over the epochs, indicating that the model is improving its latent space imputations. On the other hand, $\mathcal{L}_{\text{domain}}$ is expected to increase, with the domain classification accuracy converging towards random chance ($\frac{1}{L}$, with L equal to the number of domains), reflecting both the struggle of the domain classifier in distinguishing between the domains, and the success of the encoder in learning domain-invariant features. Since $\mathcal{L}_{\text{model}}$ reflects the adversarial min-max game between the encoder and the domain classifier (Eq.4.13b) oscillatory behavior is expected. As for the task-specific loss, $\mathcal{L}_{\text{task-specific}}$, and the reconstruction loss, $\mathcal{L}_{\text{reconstruction}}$, a decreasing trend is expected throughout the training process.

As the central objective of the proposed model is imputation, we directed our attention to two key metrics that can give insight into the imputation capability of our model: the RMSE and the Pearson correlation coefficient. Since by the end of each fold our model is trained, we assessed the correlation between the measured values (from the original data) and the predicted values (from our model) across each sample of the test set. Since the original data were normalized prior to training, we applied the inverse transformation to our predictions to return to the log-scale space of the original data (see Section 4.1.1). The comparisons were only carried out on the observed entries. Within each fold, RMSE and Pearson correlation were aggregated by domain and subsequently averaged. After completion of all folds, the RMSE and Pearson coefficients were averaged once more across folds, yielding a more reliable estimate of the generalization performance of our model.

In addition to that, we further examined potential batch effects. Prior to imputation, the datasets were visualized using a UMAP representation to assess whether samples exhibited domain-driven clustering. Given the adversarial nature of the model, the encoder is expected to mitigate these batch effects by learning domain-invariant representations. To evaluate whether these batch effects were mitigated, we inspected the latent space once again with a UMAP representation after training the model.

4.4 Inference

Once the model has been trained, it can be used to perform imputation on unseen proteomics datasets. The input of the model for inference is a protein abundance matrix formatted as a CSV file where rows

correspond to samples and columns correspond to proteins, consistent with the training input format. Unlike training, no "Domain" column is required, since domain labels are only necessary to enforce the adversarial objective during training. If the model was trained with log-transformed data, the same transformation is expected for inference data to ensure consistency. Conversely, if non-transformed data were used for training, the inference data should also be non-transformed.

During inference, the domain-adversarial components (domain classifier and GRL) are disabled, leaving only the modules responsible for imputation active (encoder, GAIN and the decoder). As such, the model focuses exclusively in reconstructing the missing protein intensities, while the representations remain domain-invariant as learned during training.

The resulting matrix $\hat{X}$ contains both the observed entries and the imputed values for the initial missing entries. Since the model operates on normalized data, the inverse transformation of the scaler used during training must be applied to return the data to its original protein space. Finally, the imputed dataset is stored as a CSV file with the samples as rows and proteins as columns under a `Result` directory.

4.5 Hyperparameters Optimization

The hyperparameter optimization was carried out using Optuna, an automatic optimization framework [27]. To balance computational cost and reliability, a two-fold cross-validation strategy was employed throughout the optimization process. A single optimization objective was defined to minimize the average best validation RMSE across folds. This metric was chosen since it directly reflects the imputation quality of the model, which is the ultimate goal of this work.

To reduce computational time, the optimization was performed on a smaller subset of the HeLa dataset, described in Section 5.1.1, using only the last 1,000 proteins. However, it is important to note that by using a reduced subset it may introduce some degree of bias. To further limit computational time, the maximum number of epochs was limited to 100, although this may not allow full convergence, the influence of each hyperparameter remains sufficient to guide the optimization.

The search covered the hyperparameters listed in Table 4.1, and a total of 50 trials were executed. The optimization took approximately 30 hours on a NVIDIA RTX A4000 GPU with 16 GB of RAM. It is important to highlight that the search for the loss weights respected the constraint $\beta, \alpha > \gamma$ (see Section 4.2).

Although hyperparameter optimization is ideally performed prior to the experiments, it is both computationally and time intensive. Given that the model development followed an iterative process, we chose to perform the hyperparameter optimization as the last analysis. Throughout the experiments, the hyperparameters were manually adjusted. This manual selection is inherently less exhaustive than the

Hyperparameter	Search Space	Optimal Value
Batch size	$\{16, 32, 64, 128\}$	16
Learning rate	$[10^{-5}, 10^{-4}]$ (log scale)	3.16×10^{-5}
Weight decay	$[10^{-5}, 0.5]$ (log scale)	0.03
Number of hidden layers	$\{1, 2, 3\}$	2
Dropout rate	$[0.1, 0.6]$ (step of 0.1)	0.3
GAIN loss weight (α)	$\{\max(1, \lceil \gamma \rceil + 1), \dots, 3\}$	3
Reconstruction error weight (β)	$\{2, 3, 4, 5\}$	3
Domain loss weight (γ)	$[0.5, 1.0]$ (step of 0.1)	1

Table 4.1: Optimal hyperparameters and hyperparameters search space provided to Optuna. Unless otherwise stated, increments are one integer unit.

optimization done by Optuna, as such it is reasonable to expect that the optimal hyperparameters would yielded more pronounced results.

Notably, the manually selected values were relatively close to the optimal ones. We consistently chose two hidden layers for the encoder and decoder, a dropout rate of 0.3, a β of 3, and a learning rate of 1×10^{-4} , compared to the optimal learning rate of 3.16×10^{-5} . The optimization, however, indicates that for optimal performance, the GAIN loss and the reconstruction error should contribute equally to the model loss, suggesting that both the corresponding modules are equally important for imputation quality ($\alpha = \beta$). This is where the difference with the manual selected values is more sound, where the α was consistently set to 2.

4.6 Imputation with a pretrained model

It is well understood that proteins are biological entities shared across experiments and conditions, in this work, we hypothesized that models trained on one dataset can potentially capture transferable protein-level representations to use on other datasets, thereby improving the imputation process.

To this end, we consider the source dataset as the data on which the model is initially trained, and the target dataset refers to the dataset to be imputed. In all experiments, the pretraining and fine-tuning were conducted using our proposed GAIN-DANN model.

We evaluated three fine-tuning strategies, each of increasing complexity, to transfer the pretrained knowledge to the target dataset. For all strategies, the imputations were restricted to the intersection of proteins present in both the source and the target dataset. To preserve the dimensionality expected by the trained model and avoid discarding learned representations, the target dataset was subsequently padded to match the source dimensionality, with the proteins not present in the target dataset repre-

sented as *NaN*.

4.6.1 Full Encoder Freezing

In an initial stage, we froze the encoder entirely and retrained only the decoder and the GAIN module on the target dataset. This approach assumes that the pretrained encoder already captures a sufficiently general representation of the protein features.

4.6.2 Partial Encoder Freezing

The second strategy unfreezes the last two layers of the encoder. This is motivated by the learning patterns of feedforward neural networks, where the initial layers tend to capture more general patterns whereas the last layers capture more dataset-specific characteristics. By fine-tuning the last two layers, the model can adapt its representations to the target dataset without discarding the general knowledge obtained from the source dataset. Since more parameters are now updated, this strategy was trained for a higher number of epochs compared to the Full Encoder Freezing strategy.

4.6.3 Domain-Adversarial Adaptation

The third strategy fully exploits the adversarial architecture of our model. Instead of only retraining on the target dataset, we provide both the source and target datasets to the model during fine-tuning. That way, the model is retrained with the objective of minimizing the discrepancies between the source and target datasets, allowing the learned representation to be adapted taking into consideration the target distribution.

In order to accommodate this setup, the model must distinguish among a larger number of domains. Consequently, the domain classifier module is reinitialized, with the number of output classes updated to *total number of domains = number of source domains + number of target domains*. To avoid hindering the encoder’s learning process, the hidden dimension of the domain classifier was set to 64. Apart from reinitializing the domain classifier, the model architecture remained the same as the pretrained architecture. During fine-tuning, the parameters of all trainable modules were updated to adapt to the target dataset.

4.7 Implementation and Usage Details

Although the core contribution of this work is methodological, we also provide an implementation designed to facilitate practical usage, in particular for imputation with a pre-trained model, which is one of

the main contributions of this thesis. To this end, the framework is structured around two main components: (i) a parameter configuration file, and (ii) a set of command-line arguments that define the execution mode of the model. All the implementation of this work as well as detailed instructions for execution are provided in the public repository¹. Furthermore, as part of a collaborative effort, a thesis developed in parallel within the same project has made a pre-trained version of the model publicly available on the Hugging Face platform² to facilitate its use and testing by the community.

Parameter configuration file. The framework relies on two JSON configurations that define the model setup:

- **Training configuration file** used for model training. It specifies both hyperparameters (number of layers, hidden dimension, learning rate, batch size, weight decay, dropout probability and the loss weights), dataset related parameters (dataset path, missing rate), and finally the training parameters (number of epochs, number of folds, random seed for reproducibility, and the hint rate for the GAIN component of our model). If any parameter is omitted in the configuration file, the optimal hyperparameters, specify in Section 4.5, values are used.
- **Inference configuration file** used both for imputation and fine-tuning. This file contains the path to the trained model and the path to the dataset to be imputed.

Both these configuration files have a `mode` parameter to specify whether we are in training mode or inference mode. This parameter is only relevant for implementation purposes to validate the parameters. In the training configuration file this variable is set to "train" while in the inference configuration file is set to "inference". Templates files for both the configurations are provided in the repository. This design helps achieve a separation between experimental settings from the source code, allowing users to easily adapt the framework to train on new datasets, and impute on new datasets with the pre-trained models.

Execution modes. The implementation exposes a single entry point through a main script, which can be executed in different modes depending on the task:

- `train`: trains a new model based on the training configuration file.
- `save`: save the trained model, this argument should be provided alongside `train` if the model is to be stored.
- `impute`: performs imputation on a new dataset using a trained model.
- `finetune`: adapts a pre-trained model to a new target dataset through one of the available fine-tuning strategies described in Section 4.6.
- `total`: uses the Full Encoder Freeze strategy (Section 4.6.1).

¹<https://github.com/QuantitativeBiology/GenerativeProteomics>

²https://huggingface.co/QuantitativeBiology/GAIN_DANN_model

- `partial`: uses the Partial Encoder Freeze strategy (Section 4.6.2).
- `total`: uses the Domain-Adversarial adaptation strategy (Section 4.6.3).

5

Results and Discussion

Contents

5.1 Datasets	43
5.2 Evaluation	50
5.3 Discussion	59

This section presents the results obtained with the proposed model and discusses its performance in proteomics data imputation. Section 5.1 provides a detailed description and analysis of the datasets used throughout the experiments. Section 5.2 presents the results and analysis of both intra-dataset and cross-dataset experiments. Finally, Section 5.3 discusses the benefits and limitations of the proposed model.

5.1 Datasets

In the following sections, we describe and analyze the datasets used for the development of our model 5.1.1 and the datasets used to evaluate imputation with a pretrained model 5.1.2.

5.1.1 HeLa dataset

A HeLa dataset, provided PRIDE¹ at EMBL-EBI, was used throughout the development of our model [9]. This dataset was provided in an 'anndata' format with multiple layers corresponding to different transformations of the data. Among these, we selected the log-transformed (*lbaqLog*) layer. This choice was motivated by two considerations: the raw intensity values are scattered over several orders of magnitude ($[0, 10^{10}]$) making them unfeasible to feed to the model, besides that log-transformed data is the typical choice in a proteomics settings.

After selecting the *lbaqLog* layer, the dataset was represented as a matrix of size $4,820 \times 11,013$, with rows corresponding to samples and columns corresponding to proteins. Each of the samples is uniquely identified, following the format <Project ID>-Sample-<Sample Number>, linking it to its corresponding project. These project identifiers were used as the sample domain, necessary for our model, resulting in 17 distinct domains.

Following that, a comprehensive analysis of the HeLa dataset was conducted.

1. Distribution of protein intensities values. The distribution of nonzero protein intensities in the HeLa dataset follows an approximately bell-shaped curve, centered around the log-intensity values of 5 and 7 (Fig. 5.1).

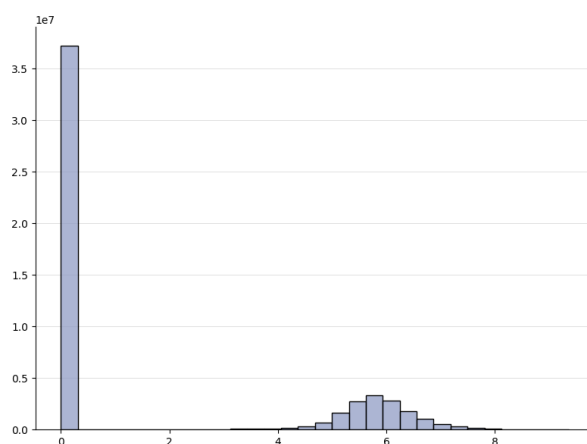


Figure 5.1: Distribution of protein intensities values in the HeLa dataset (*lbaqLog* layer).

The protein intensities distributions across projects are highly homogeneous, exhibiting a mean of 5.82 ± 0.74 (mean $\pm$ std) (Fig. A.1).

2. Zeros as missing values. The distribution of values is not homogeneous throughout the whole range of values. It is important to note that the total number of zeros (10^7) is an order of magnitude higher than the rest of the values (10^6). Moreover, the zeros cluster at the tail of the distribution, suggesting that they do not arise from the same distribution (5.1). Given these observations and the inherent

¹<https://www.ebi.ac.uk/pride/>

characteristics of mass spectrometry, where missing values often occur, we interpreted the zeros as missing values rather than as a true absence of expression. This resulted in an overall missingness of approximately 70%. Despite the considerable level of incompleteness, the dataset remains sufficiently large to support the model development.

3. Distribution of missing values. After the replacement of zeros as *NaN*, we analyzed the distribution of missing values across samples, proteins, and projects. A significant portion of the samples have moderate to high missingness. In fact, half of the samples (2,412) have more than 71.3% missing protein intensities (Fig. 5.2(a)). The distribution of the missing values in samples is strongly right-skewed, with only 3 samples exhibiting missingness lower than 20%, all from project PXD013658, and 225 samples have missingness between 20 and 50%, the majority belonging to project PXD048325, while the majority (3,915) of samples have around 65 – 75% of missingness, the majority of them belonging to project PXD042233. At the protein level, the missingness follows a bimodal distribution, suggesting that the missingness is not random but likely related to detection sensitivity in mass spectrometry. This reflects the polarized nature of missingness in proteomics data: proteins are either well-detected or poorly detected (Fig. 5.2(b)). No protein was entirely missing across all the samples or within the samples of any project. Proteins with high missingness exhibit systematically lower intensities (Fig. 5.3). More than half of the proteins (6,661, or 60%) exhibit missingness above 90% (Fig. 5.2(b)). Proteins in this group also display systematically lower observed intensities with a mean of 4.31 compared to low missing proteins (< 1% missing) with a mean of 6.19 (Fig. 5.3). These results highlight the difficult task of proteomics imputation where the majority of the missing values occur in low-abundance proteins. Missingness across projects varied substantially, ranging from 20% up to 86% (Fig. 5.4). Since the missingness does not scale directly with sample size, the variation is likely explained by different experimental setups. Most projects exhibit a missingness higher than 60% consistent with the typical behavior of large-scale proteomics datasets, highlighting the need for robust imputation strategies.

4. Dataset imbalance. As previously stated, the HeLa dataset has 4,820 samples distributed across 17 different projects, each sample containing measurements of 11,013 protein intensities. However, the dataset is highly imbalanced (Table 5.1). Project PXD042233 alone accounts for 88% of all samples (4,258 out of 4,820), whereas the majority of the projects have less than 50 samples.

Any imbalance representation of the domain classes, not exclusively in this dataset, can bias the learning process towards domain-specific characteristics rather than promoting domain-invariant representations. This was taken into account in the training process. In order to both prevent the bias and to preserve the domains proportions, a weighted sampler was employed during the data splitting. We further investigated the imbalance influence by conducting experiments with and without PXD042233 (see Section 5.2.1).

5. Batch effects. Since all samples derive from the same biological source (HeLa cells), substantial

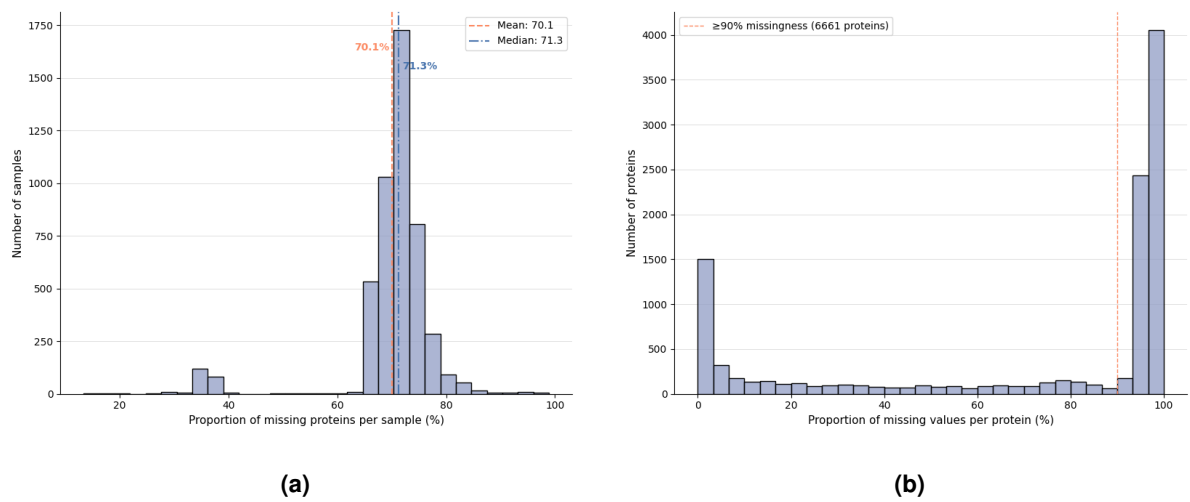


Figure 5.2: Distributions of missingness (a) across samples, (b) and across proteins 6,661 proteins have a missingness higher than 90% in the HeLa dataset.

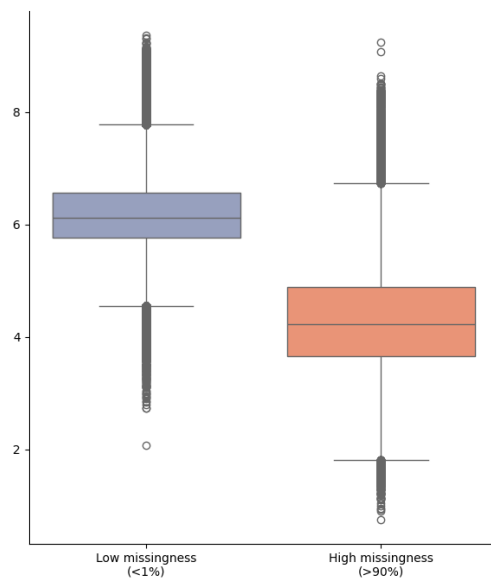


Figure 5.3: Distribution of protein intensities for low-missingness (<1%) proteins ($n = 858$ proteins) and high-missingness (>90%) proteins ($n = 6,661$ proteins). The mean protein intensities for the low-missingness group is 6.19, and for the high-missingness is 4.31.

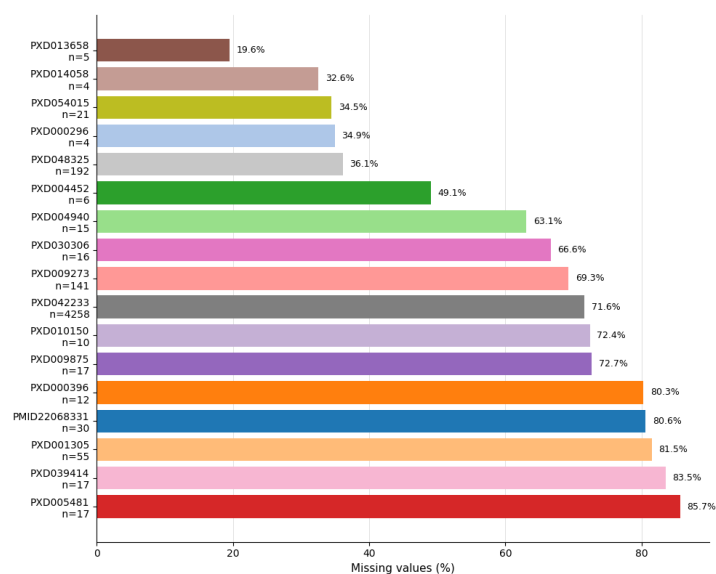


Figure 5.4: Percentage of missing values per sample across individual projects within the HeLa dataset. Each bar represents one project, with the corresponding number of samples indicated below the project ID (n = number of samples).

Project	Number of Samples
PXD014058	4
PXD000296	4
PXD013658	5
PXD004452	6
PXD010150	10
PXD000396	12
PXD004940	15
PXD030306	16
PXD009875	17
PXD039414	17
PXD005481	17
PXD054015	21
PMID22068331	30
PXD001305	55
PXD009273	141
PXD048325	192
PXD042233	4,258

Table 5.1: Number of samples per project in the HeLa dataset.

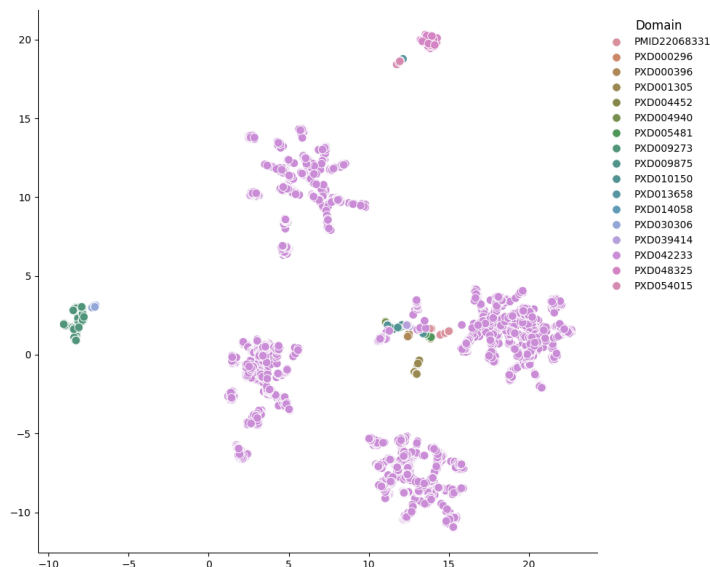


Figure 5.5: UMAP projection of the HeLa dataset colored by project. Samples cluster strongly by project, indicating batch effects.

overlap of the samples across projects in a UMAP would be expected. Instead, the UMAP projection reveals a clear clustering by project rather than biological variation, revealing that the dataset suffers from batch effects (Fig. 5.5). The presence of these batch effects provided further motivation for incorporating domain-adaptation strategies to mitigate project-specific biases and to promote learning of domain-invariant representations.

In summary, the HeLa dataset is characterized by a high overall proportion of missing values, with a bimodal distribution at the protein level reflecting the detection sensitivity of mass spectrometry. Additionally, missingness is not uniformly distributed: some proteins and projects are systematically more incomplete than others. The dataset also displays a strong imbalance with respect to the projects, with one project (PXD042233) containing the majority of the samples. The HeLa dataset also exhibits batch effects, as evidenced by project-specific clustering in the UMAP projection. These observations provided motivation for employing domain-adaptation strategies with the adversarial training, to promote domain-invariant representations and more robust imputations across heterogeneous projects.

5.1.2 PRIDE datasets

We also had multiple PRIDE proteomics datasets of different tissues types available provided by the EMBL-EBI (Table 5.2). These datasets were used to evaluate the cross-domain scenarios described in Section 4.6.

Project	Tissue	Number of Samples	Number of Proteins	Missing Rate (%)
PXD005736	Heart	2	8,470	2.10
PXD002179	Testis	3	8,836	3.38
PXD012131	Brain	26	10,188	6.77
PXD005445	Brain	7	9,165	7.42
PXD008840	Stomach	84	9,278	14.24
PXD010154	Multiple	38	14,669	17.15
PXD004452	Multiple	4	8,657	17.44
PXD019909	Skin	12	11,700	18.62
MSV000085836	Multiple	460	13,051	29.62
PXD006675	Heart	56	11,135	30.81
PXD020192	Multiple	46	6,177	34.88
PXD000612	HeLa-S3	11	8,782	39.62
PXD030304	Multiple	2,012	8,754	39.93
PXD000865	Multiple	34	8,680	60.51
PXD000561	Multiple	85	12,025	80.23

Table 5.2: PRIDE datasets, ordered by ascending missing rate. Datasets annotated as "Multiple" in the Tissue Type include samples from more than one tissue type. Tissue types in bold correspond to datasets containing a single tissue type. A more detailed table regarding tissue composition is provided in Appendix A.1.

As shown in Table 5.2, the datasets have a wide range of tissue types. To assess the effect of transferring knowledge from a pretrained model to a target dataset of the same tissue type, we filtered datasets corresponding to specific tissue types (Table 5.3).

Project	Tissue	Number of Samples	Number of Proteins	Missing Rate (%)
PXD010154	Brain	1	12,297	0
PXD020192	Brain	1	4,653	0
PXD012131	Brain	26	10,190	6.77
PXD005445	Brain	7	9,167	7.42
PXD000561	Brain	2	4,090	36.12
PXD000612	HeLa-S3	11	8,782	36.85

Table 5.3: Summary of PRIDE projects after filtering and grouping samples by tissue type. The original PRIDE datasets are detailed in Table 5.2.

We then analyzed the protein intensity distributions of these filtered datasets (Fig. 5.6).

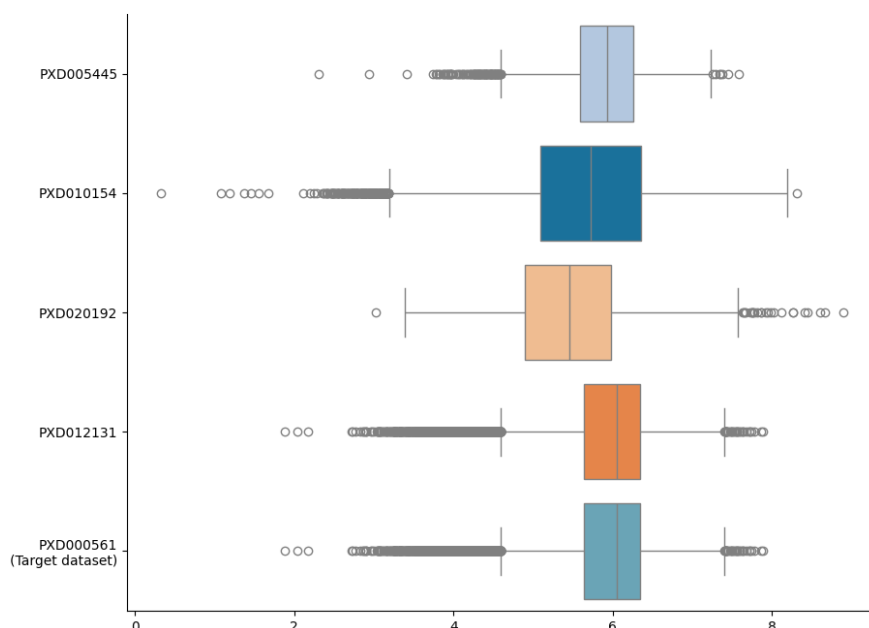


Figure 5.6: Distribution of protein intensities on the Brain across PRIDE datasets.

5.2 Evaluation

The proposed model was evaluated in two different settings:

- **Intra-Dataset** where the model was evaluated on the same dataset with multiple related domains. This assesses the model ability to learn domain-invariant representations and generalize across domains. The HeLa dataset was used since it has samples from the same biological source across multiple projects (domains).
- **Cross-Dataset** in which a pretrained model on a source dataset is used to help impute missing values in a target dataset. This assesses if pretrained representations can improve imputation performance.

All the experiments were conducted on a NVIDIA RTX A4000 GPU with 16 GB of RAM.

5.2.1 Intra-dataset

To assess the model ability to generalize across domains within the same biological source, we conducted an intra-dataset evaluation using two variants of the HeLa dataset: **HeLa-Reduced** and **HeLa-**

Reduced-NoPXD042233.

Due to computational limitations, the HeLa-Reduced dataset was obtained by excluding the 2,000 proteins with the highest proportion of missing values, resulting in 4,820 samples and 9,013 proteins and an overall missingness of 63.43%. Since the project PXD042233 accounts for the majority of the samples, we repeated the same experiments after excluding it, to assess whether by having more samples it enforces more pressure on the model to distinguish it from the other projects, potentially influencing the learned latent representation. This new variant, referred to as HeLa-Reduced-NoPXD042233, comprises 562 samples across 9,013 proteins and an overall missingness of 58.65%.

A GAIN-DANN model was trained in each dataset for 200 epochs, with an early stopping patience of 10 epochs, with the following hyperparameters: learning rate of 1×10^{-4} , two hidden blocks for both the encoder and decoder, dropout rate of 0.3, $\alpha = 2.0$, $\beta = 3.0$ and $\gamma = 0.5$.

For evaluation, an additional 10% of the observed protein intensities were randomly masked in each dataset. This modified dataset was used to assess the performance of both the GAIN-DANN model and a baseline ProtoGain model. The baseline model was trained with all the default hyperparameters. All reported results correspond to the models performances on the datasets with the additional 10% induced missingness.

HeLa-Reduced

Since all samples are derived from the same biological source (HeLa cells), a substantial overlap among samples from different projects would be expected in a UMAP representation. Instead, the UMAP projection reveals a clear clustering by project revealing that the dataset suffers from batch effects (Fig. 5.7(a)). One of the implicit objectives of our model is to learn domain-invariant features through the encoder so that the learned representation can generalize across domains. Consequently, we expect that our model can mitigate the batch effects.

Indeed, the UMAP projection of the latent space, after the training of our model, shows that batch effects were almost completely mitigated (Fig. 5.7(b)). Two distinct clusters can be observed: the rightmost cluster contains samples from multiple domains, suggesting that it no longer reflects project identity, while the leftmost cluster consist exclusively of samples from PXD048325.

To understand why the project PXD048325 remains distinguishable, we compared the distribution of protein intensities between this project and the remaining ones. Project PXD048325 exhibits a mean intensity of 4.90 ± 1.04 (mean $\pm$ std), whereas the other projects have an average mean of 5.91 ± 0.64 (Table 5.4). Interestingly, another project, PXD054015, is also quite distinct from the others with a mean of 4.86 ± 1.09 , despite clustering with the main cluster. Both projects present relatively low missingness rates, PXD054015 has 20% and PXD048325 has 22%, but differ significantly in the sample size, with the former having only 21 samples versus 192 samples for the PXD048325.

To further investigate whether the PXD054015 could be influenced by PXD042233, the largest

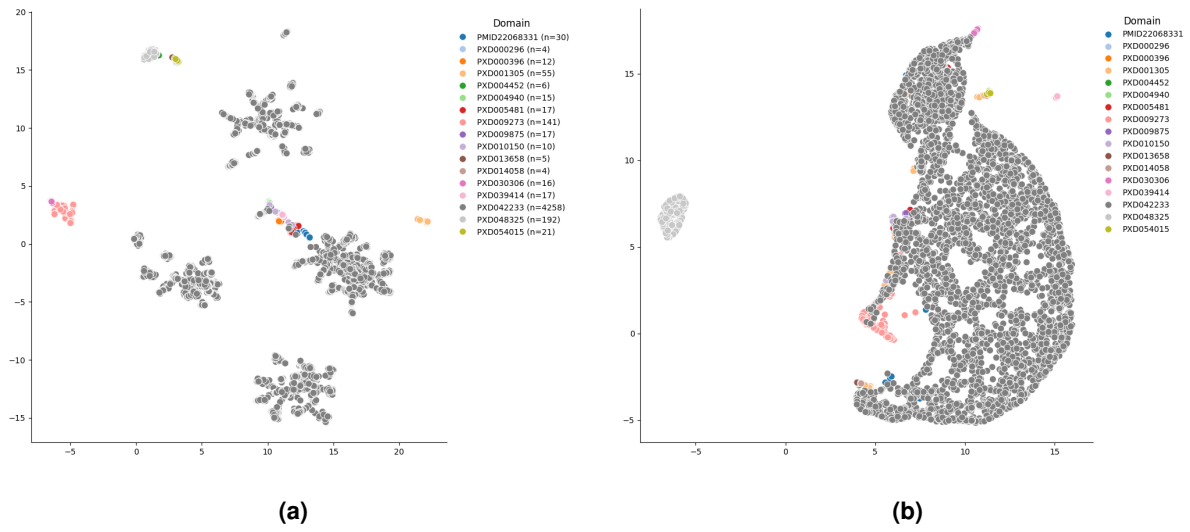


Figure 5.7: UMAP projection (a) of the HeLa-reduced, (b) of the latent representation of the HeLa-Reduced after training of our model.

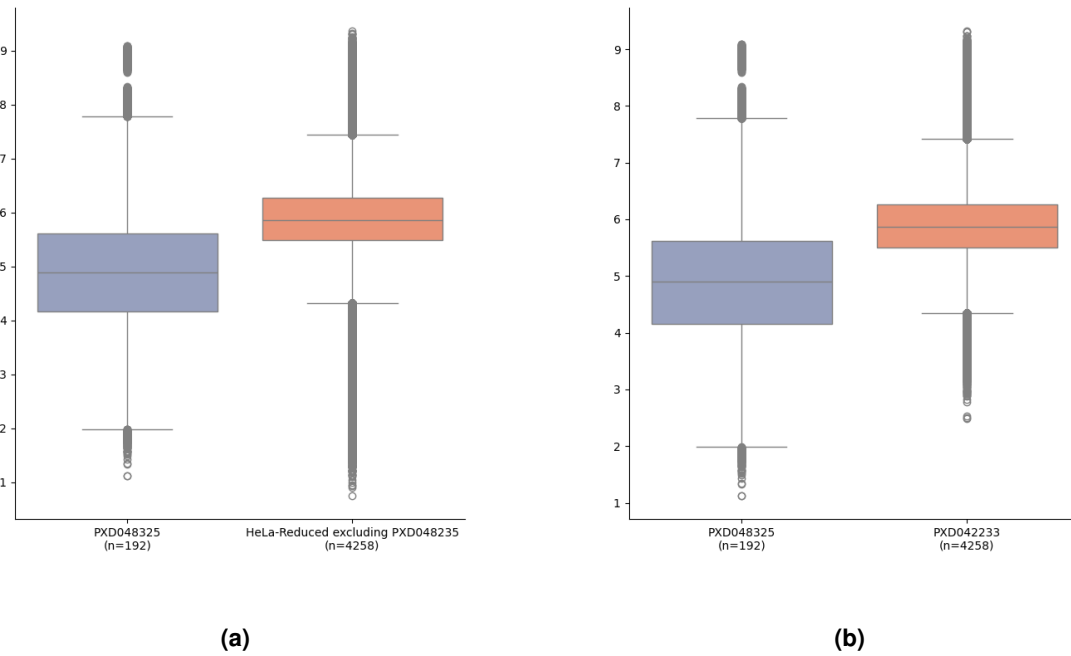


Figure 5.8: Distribution of protein intensities of the HeLa-Reduced dataset (a) PXD048325 (4.90 ± 1.04) compared to the full HeLa-Reduced (5.91 ± 0.64), (b) PXD048325 compared to PXD042233 (5.12 ± 0.61).

Project	Mean $\pm$ Std
PXD005481	6.28 $\pm$ 0.64
PXD039414	6.26 $\pm$ 0.58
PXD000396	6.12 $\pm$ 0.64
PXD001305	6.09 $\pm$ 0.67
PXD009273	6.04 $\pm$ 0.55
PMID22068331	5.93 $\pm$ 0.85
PXD042233	5.92 $\pm$ 0.61
PXD009875	5.87 $\pm$ 0.69
PXD030306	5.87 $\pm$ 0.63
PXD010150	5.77 $\pm$ 0.74
PXD004940	5.75 $\pm$ 0.64
PXD000296	5.35 $\pm$ 0.76
PXD004452	5.24 $\pm$ 0.91
PXD014058	5.16 $\pm$ 0.80
PXD013658	5.13 $\pm$ 0.83
PXD048325	4.90 $\pm$ 1.04
PXD054015	4.86 $\pm$ 1.09

Table 5.4: Distribution of the Mean and Standard Deviation of the protein intensities of each individual project of the HeLa-Reduced dataset ordered by increasing Mean value.

project, we quantified the similarity of their protein coverage patterns. For each project, a coverage vector was obtained by calculating for every protein the fraction of samples with non-missing abundance values. Let $C_i \in [0, 1]^m$ denote the coverage vector of project i , where each element represents the proportion of non-missing values for a given protein. The similarity between two projects, i and j , was then defined as the Pearson correlation coefficient between their coverage vectors (Equation 5.1).

$$\text{sim}(i, j) = \text{corr}(C_i, C_j) \quad (5.1)$$

The similarity coefficients $\text{sim}(\text{PXD054015}, \text{PXD042233}) = 0.429$ and the $\text{sim}(\text{PXD048325}, \text{PXD042233}) = 0.456$ indicate that both projects exhibit a similar coverage with respect to PXD042233. Thus the coverage similarity alone does not explain the clustering pattern. A more plausible explanation is that the limited number of samples in PXD054015 constrains its representation in the latent space, and that with more samples, this project might had diverge further from PXD042233, resembling the observed pattern for PXD048325.

As for the imputation quality, we analyze the average Pearson correlation and RMSE across folds for each individual project (Figures 5.10(a) and 5.10(a)). As anticipated, projects with lower Pearson coefficients generally coincided with higher RMSE values. The Pearson correlation quantifies the linear association between predicted and observed protein intensities in the original protein space, whereas the RMSE captures the reconstruction error in the standardize space. Jointly, these metrics provide a

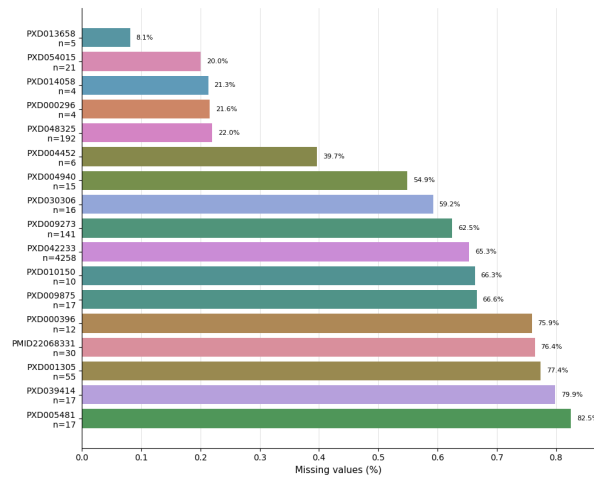


Figure 5.9: Percentage of missing values per sample across individual projects of the HeLa-Reduced dataset. Each bar represents one project, with the corresponding number of samples indicated below the project ID (n = number of samples).

more comprehensive view of the model performance.

A clear trend emerges with respect to the sample size. Larger projects, such as PXD048325 ($n = 192$) and PXD009273 ($n = 141$) achieved the best overall performance, with Pearson correlations above 0.96 and RMSE values below 0.22, indicating that the imputations are highly accurate. In contrast, smaller projects, particularly those with fewer than 10 samples (e.g., PXD014058, and PXD004452), exhibited substantially weaker performance, with RMSE values ranging between 0.76 and 0.858 and Pearson correlations lower than 0.6. This possibly reflects that having a limited sample representation hinders the model ability to capture project-specific characteristics important for the learning task: imputation.

Nevertheless, the sample size alone does not fully explain the variability. For instance, project PXD000296 with only 4 samples achieved a Pearson correlation of 0.907, outperforming other small sample projects with smaller missingness, namely PXD014058 with 4 samples and an overall missingness of 8.1% obtained only a Pearson of 0.513 and a RMSE of 0.858.

The aggregated values of the Pearson correlation and RMSE provide a global assessment of the model performance across the projects. While scatter plots of the original values versus the predicted values, enable a more granular sensitivity of the imputation quality at the sample level. Figure 5.11(a) illustrates the relationship between the original and imputed protein intensities for the sample with lowest performance within the PXD004452. In this sample, the correlation between the original sample and the imputed sample is 0.57 and the RMSE is 0.90, aligned with the relatively lower performance of this project.

The distribution of points around the diagonal reveals that the imputed values generally follow the true intensities, but deviations are more pronounced at higher intensity ranges, suggesting a tendency

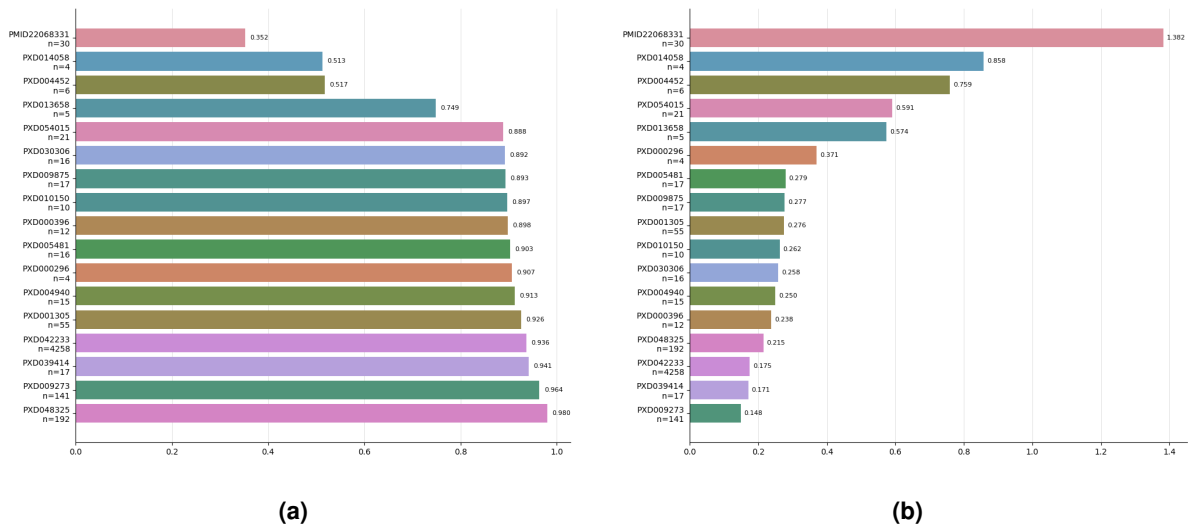


Figure 5.10: Imputation quality. Results for each individual project on the HeLa-Reduced dataset after training the GAIN-DANN model with a two-fold cross-validation (a) Pearson correlation coefficient, (b) RMSE. Each bar represents one project, with the corresponding number of samples indicated below the project ID (n = number of samples).

towards underestimation. Following the same trend, the scatter plots from projects with higher performance consistently display a tighter clustering around the diagonal, reflecting more accurate imputations (Fig. 5.11(b)).

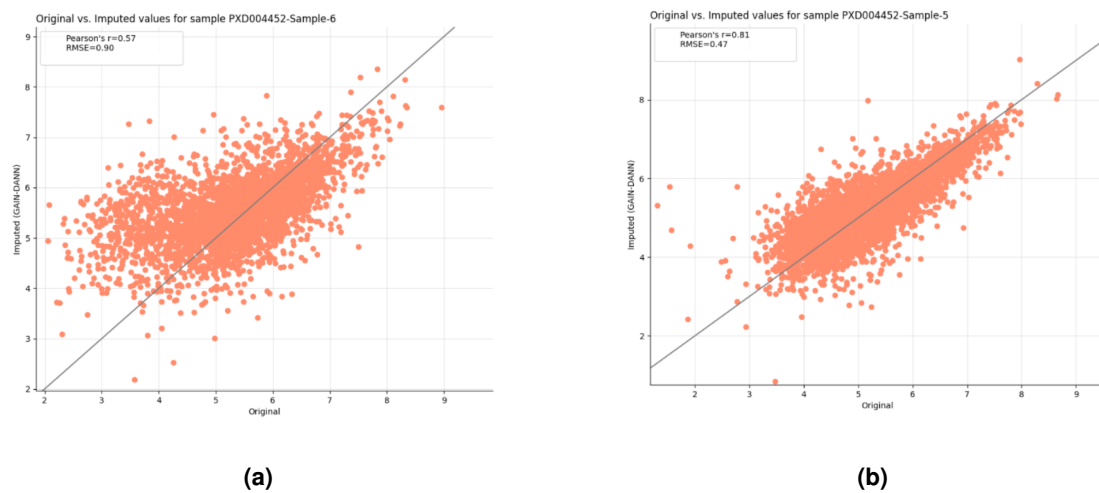


Figure 5.11: Scatter plots showing the correlation between observed and imputed protein intensities for two samples from project PXD004452 of the HeLa-Reduced dataset (a) sample with the lowest performance, (b) sample with the highest performance.

Overall, the GAIN-DANN model demonstrated strong imputation performance on the HeLa-Reduced

dataset, with an average Pearson correlation of 0.83 ± 0.18 and an average RMSE of 0.42 ± 0.32 , taking approximately 10 hours to train. Its performance is comparable to the performance of the baseline ProtoGain model, which achieved an RMSE of 0.73 and required approximately 8 minutes to train. However, it is important to note that the evaluation of the GAIN-DANN model is more rigorous than the validation of the ProtoGain. At the time of its development, reduced data availability constrained ProtoGain to be trained on all the data availability while at the same time being evaluated on a portion of that data, thus this can lead to overfit (Section 3.2). In contrast, the GAIN-DANN model was assessed using a cross-validation strategy.

HeLa-Reduced-NoPXD042233

Since the project PXD042233 accounts for the majority of the samples in the HeLa-Reduced, we wanted to inspect whether it enforces more pressure on the model to distinguish it from the other projects, potentially influencing the learned latent representation.

In reality, that does not seem to happen. The UMAP projection of the latent space seems to aggregate the samples in five main clusters (Fig. 5.13(b)). Interestingly, the projects with a more distinct distribution of protein intensities (PXD048325 and PXD054015), form their own cluster. This confirms the earlier suspicion that these datasets are indeed different from the remaining projects, possibly reflecting differences in experimental settings.

Additionally, projects with lower missingness, such as the project PXD013658, PXD014058 and PXD000296, are also clustered together in the lowest right side of the UMAP. The grouping aligns with their coverage similarity ranging from 0.45 to 0.55 (Fig. 5.12).

This leads us to wonder if the latent representation captures protein completeness instead of protein abundance. However, the Spearman correlation between the pairwise coverage similarity and the latent space distance between the centroids of the projects, represented as domain in Fig. 5.13(b), was only -0.267 suggesting that there is only a moderate inverse relation. In contrast, the correlation between abundance similarity and latent distance between the centroids was -0.058 suggesting that the clustering in the latent space is not only explained by the abundance distribution. As such, the encoder appears to capture other sources of variation beyond protein coverage and abundance, it may possibly reflect biological variation or different experimental settings.

Overall, the model achieved similar performance to the HeLa-Reduced setting, suggesting that removing PXD042233 did not impact the imputation quality neither the model's generalization (Figures 5.14(a) and 5.14(b)).

5.2.2 Cross-Dataset

To assess whether pretrained representations can improve the imputation of missing values, we conducted a cross-dataset evaluation using the transfer learning strategies described in Section 4.6.

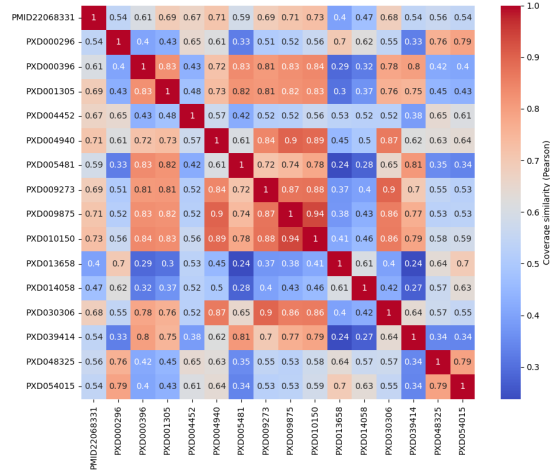


Figure 5.12: Coverage similarity heatmap for the HeLa-Reduced-NoPXD042233 dataset. The values represent the pairwise Pearson correlations between the protein coverage vectors of the HeLa-Reduced-NoPXD042233 projects.

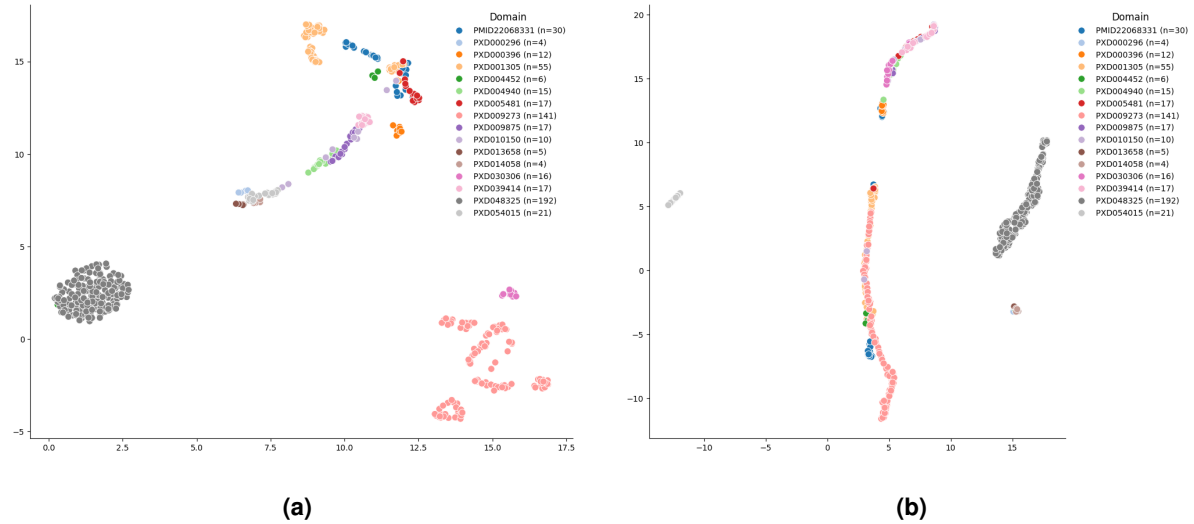


Figure 5.13: UMAP projection (a) of the HeLa-Reduced-NoPXD042233, (b) of the latent representation of the HeLa-Reduced-NoPXD042233 after training of our model.

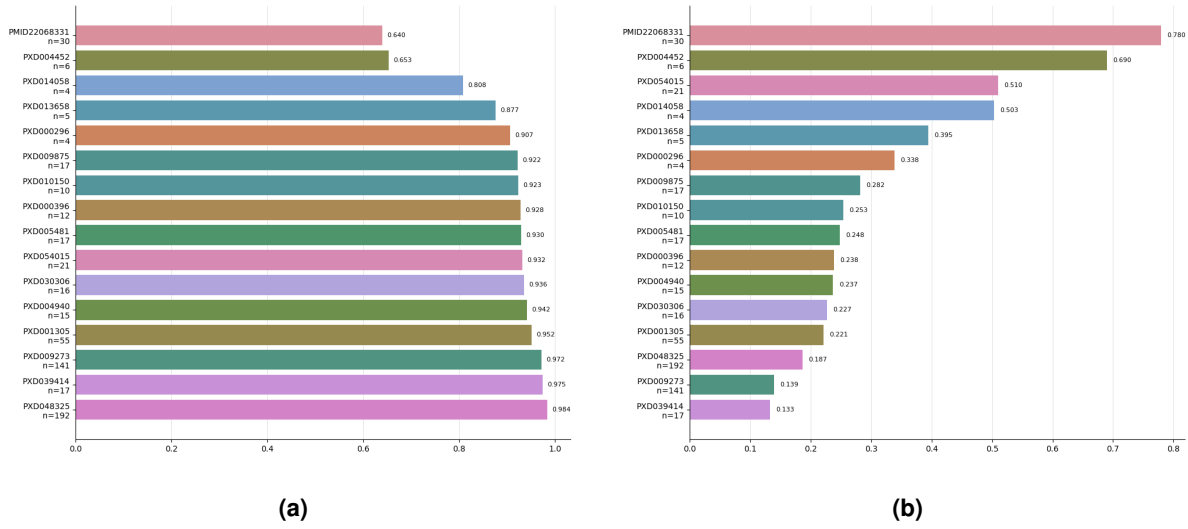


Figure 5.14: Distributions of missingness across samples (a) and proteins (b) in the HeLa-Reduced-NoPXD042233 dataset.

We pretrained models on datasets of the same tissue type, leaving the dataset with the highest missingness within that tissue type as the target (Table A.1). This design reflects the intuition that small and highly incomplete datasets should benefit the most from transfer knowledge compared to models trained solely on that limited target data. In addition, most of the source datasets exhibited very low levels of missingness, in some cases close to zero, which makes them ideal candidates for knowledge transfer. To better understand whether the knowledge transfer indeed helped imputation quality, a baseline ProtoGain model was also trained directly on the target dataset. If the retrained model with the transfer knowledge outperformed the baseline, we can then argue that the transfer knowledge was indeed beneficial. Conversely, if the baseline model yielded better results, conclusions would need to be drawn more cautiously, since very small target datasets are prone to overfitting, as discussed in Section 3.2.

The source dataset for each tissue was constructed by merging all samples belonging to the same tissue type. Proteins absent from individual projects were set to *NaN* in the union dataset. To maximize performance, we restricted the union dataset to the intersection of proteins shared with the target dataset. This ensured that the pretrained model would focus on imputing proteins relevant to the target dataset. In all cases, the domain of each sample corresponds to the PRIDE project it belonged to.

All models were pretrained on the source dataset for 200 epochs, with an early stopping patience of 10 epochs and the following hyperparameters: learning rate of 1×10^{-4} , two hidden layers for both the encoder and decoder, a dropout rate of 0.3, $\alpha = 2$, $\beta = 3$ and $\gamma = 1$. During fine-tuning, the same hyperparameters were used. In addition, the domain adaptation weight λ_p was fixed at 0.2 in all experiments. This value was chosen empirically as it provided the most stable results, ensuring that the

adversarial contribution remained constant across training epochs and thus facilitated the adaptation to the target dataset. This aligns with the objectives of the fine-tuning stage that was designed to take advantage of the pretrained model to reduce computational time while improving imputation quality.

Brain tissue type

For the brain tissue, the source dataset was constructed by merging samples from projects PXD010154, PXD020192, PXD012131, and PXD005445, resulting in a dataset with 35 samples and 13,273 proteins. The project PXD000561 was chosen as the target dataset since it exhibited the highest missingness among the brain datasets (Table 5.3). Restricting both the source and the target dataset to their protein intersection yielded 4,034 common proteins, leading to an overall missingness of 6.79% for the source dataset and 35.96% for the target dataset. Pretraining on the source dataset required approximately 31 seconds.

The results are consistent with our expectations. For the first strategy (see Section 4.6.1), where the encoder is completely frozen and only the decoder and GAIN module are retrained, the model achieved an RMSE of 1.489 after 60 epochs. Allowing partial unfreezing of the encoder and decoder yielded an RMSE of 1.202 after 100 epochs (see Section 4.6.1). The higher number of epochs in the second strategy is motivated by the higher number of trainable parameters.

Finally, the domain-adversarial adaptation strategy provided the strongest results (see Section 4.6.3). In this configuration, the model achieved an RMSE of 0.311 on the target dataset within only 28 seconds, whereas training ProtoGain directly on the target dataset required 1 minute and 18 seconds and achieved a higher RMSE of 0.946 (Table 5.5). These results support the hypothesis that small datasets with high missingness can particularly benefit from pretrained representations.

Tissue	Dataset		RMSE		Time	
	Source	Target	GAIN-DANN	ProtoGain	GAIN-DANN	ProtoGain
Brain	Union Brain	PXD000561	0.311	0.946	28s	1m18s

Table 5.5: Comparison between GAIN-DANN and ProtoGain across RMSE and runtime for the Domain-Adversarial fine-tuning strategy (see Section 4.6.3).

5.3 Discussion

The proposed GAIN-DANN model integrates generative imputation and domain-adversarial learning to address high missingness and domain shifts in proteomics data. The results across both intra-dataset and cross-dataset experiments confirm that the adversarial training of the proposed model enables the model to extract domain-invariant representations, thereby improving imputation quality. These findings support the hypothesis that shared information across related domains and datasets can enhance

imputation performance.

5.3.1 Intra-dataset

The intra-dataset setting evaluates the model ability to generalize across related domains that may suffer from distributional shifts while still sharing underlying biological information. This constitutes the foundation of this scenario: if related biological information exists, the model should be able to learn domain-invariant representations of the data that can generalize beyond domain-specific biases.

In this setting, the model was evaluated on a HeLa dataset, where all the samples share the same biological source. Under such conditions, substantial biological similarity among samples would be expected. However, the UMAP projection of the HeLa-Reduced dataset revealed clear clustering by project, indicating the presence of batch effects (Fig. 5.7(a)). Although the impact of such effects is difficult to quantify in a missing-value context, they can introduce biases that negatively compromise downstream analysis.

The GAIN-DANN model successfully reduced these batch effects while achieving a lower RMSE than the baseline ProtoGain. This improvement suggests that the domain-adversarial component aided the learning of a more faithful representation of the data rather than technical noise.

The results demonstrates that even when the samples share the same biological source, domain shifts introduced by experimental conditions can still bias the imputation. Addressing domain shifts is therefore relevant not only for cross-dataset but also within datasets with the same biological source where technical variation may be present.

5.3.2 Cross-dataset

In the cross-dataset setting, the benefits are even more pronounced and relevant, as this scenario corresponds to the primary motivation for the proposed model. The results clearly demonstrated that leveraging cross-dataset information can enhance imputation quality, particularly in data-limited conditions.

Among all the fine-tuning strategies, the domain-adversarial approach achieved the best overall performance. Its superior performance can be attributed to its emphasis in minimizing discrepancies between the source and target domains during fine-tuning. By enforcing domain-invariant representations in the latent space, the encoder was able to generalize across datasets, improving the imputation quality in the target domain.

The pretrained model not only converged faster than the baseline model, but it also achieved a substantially lower RMSE. This contrast underscores the impact of transferring pretrained representations rather than learning completely from scratch in data-limited settings. In addition to the small size of

the target datasets, they also suffer from a substantial missingness, which consequently, reduces even further the data available to the model, hindering the model ability to learn robust information. In such cases, transfer learning proved to be particularly beneficial, allowing the model to leverage the structure and variability captured from the more complete source datasets to guide the imputation process.

The alternatives fine-tuning strategies performed worse than the baseline model. This indicates that mere retraining of the last layers of the encoder and/or decoder was insufficient to overcome the domain shift, and that a more in-depth adaptation is needed by full retraining. In other words, an adaptation of the latent space through adversarial alignment allowed the model to adapt to the target distribution while retaining generalizable representations.

These findings validate the hypothesis that pretrained representations can substantially improve imputation performance in small and incomplete proteomics datasets. They also suggest that pretraining on larger datasets can further expose the model to relationships between proteins that may not be present or detectable in smaller target datasets. Once learned, such representations can then be used to aid imputation in data-limited conditions.

5.3.3 Computation considerations

While the GAIN-DANN model consistently outperformed ProtoGain in terms of RMSE in both the intra-dataset and the cross-dataset scenario, this improvement came at a computational cost. For instance, in the HeLa-Reduced dataset, ProtoGain achieved an RMSE of 0.73 in approximately 8 minutes, whereas the GAIN-DANN model achieved an RMSE of 0.42 taking approximately 10 hours. This difference was, however, expected. Since the GAIN-DANN architecture incorporates the GAIN module alongside three other modules, which increase the number of trainable parameters. Furthermore, the adversarial interplay between the modules introduces even more competing objectives, which naturally have slower convergence. This difference will only be aggravated the larger the dataset, which is a common scenario in proteomics.

Nevertheless, the cross-dataset experiments demonstrated that once pretrained, the GAIN-DANN model reached convergence substantially faster than the baseline model, achieving an RMSE of 0.63 compared to 0.95 while reducing the computational time by approximately 75%. This highlights the advantage of incorporating pretrained representations that can easily adapt to new datasets with minimal additional training.

6

Conclusion

Contents

6.1 Conclusions	63
6.2 Limitations and Future Work	64

6.1 Conclusions

In this work, we addressed the problem of missing value imputation in proteomics data, a critical step to ensure reliability in the downstream analysis. Building upon previous work in our group, that adapted GAIN to proteomics, we proposed GAIN-DANN, a hybrid model that integrates Generative Adversarial Imputation Networks with Domain-Adversarial Neural Networks.

The model was designed to exploit shared information across related domains and datasets by learning domain-invariant representations capable of generalizing to new conditions. Since working with multiple domains or datasets inevitably introduces domain shifts, the model was developed to explicitly address these shifts through its adversarial learning strategy. Across experiments, GAIN-DANN effectively mitigated these shifts improving the model generalization capacity.

In intra-dataset evaluations on the HeLa dataset, GAIN-DANN outperformed the baseline ProtoGain, achieving lower reconstruction error while reducing batch effects. In cross-dataset scenarios, the improvements were even more pronounced. The GAIN-DANN model converged faster and achieved a lower RMSE than the ProtoGain model trained from scratch. This demonstrated that previously learned representations can be reused to enhance imputation in smaller and noisier datasets.

In summary, this work introduced the GAIN-DANN framework for missing data imputation in proteomics. Through its adversarial strategy, the model is able to learn domain-invariant representations, reduce batch effects and improve imputation quality across both intra and cross-dataset settings. These findings demonstrate that incorporating domain adaptation and generative imputation models can indeed enhance the imputation process.

6.2 Limitations and Future Work

While the proposed GAIN-DANN model consistently outperformed the baseline ProtoGain in terms of imputation quality, it also required longer training times. This can be viewed as a limitation, since the addition of more modules combined with the adversarial optimization dynamics can inherently slow convergence. However, it is important to note that the training time seems to be largely influenced by the size of the dataset. For instance, training on the HeLa-Reduced dataset (4,820 samples and 9,013 proteins) required approximately 10 hours, while training on HeLa-Reduced-NoPXD042233 (562 samples and 9,013 proteins) require approximately 75 minutes. In contrast, training on the brain source dataset (35 samples and 4,035 proteins) only required 31 seconds. This disparity highlights the scalability problem common when dealing with large-scale proteomics datasets. Nevertheless, the adversarial training strategy not only improved the imputation quality but also helped mitigate batch effects, which are known to compromise downstream analysis.

Moreover, once trained the learned representations can be efficiently reused for imputations in other datasets, substantially reducing the training time while achieving better results compared to training from scratch. Thus, although the initial training phase is more demanding, the generalization capability of GAIN-DANN makes it a strong candidate for imputation in smaller more incomplete proteomics datasets.

Due to time constraints, leveraging protein-protein interaction networks to improve imputation was left unexplored. By examining existing relationships between proteins, it may be possible to infer missing relationships within a dataset, and therefore estimate protein abundance in a sample. From a graph-theoretic point of view, this can be framed as a link prediction problem, where the objective is to predict the likelihood of a missing or unobserved edge between nodes in the network.

Building upon this idea, future work could investigate the integration of Graph Neural Networks to model both node-level and edge-level relationships. At the node level, regressions tasks could be used

to predict abundance values, while at the edge-level, link prediction tasks could exploit co-occurrence patterns and protein-protein relations to enhance the imputation process. This dual view would allow the module to capture both individual protein characteristics and protein relations with PPI networks. Several GNN architectures could be explored:

1. **GNN built from scratch**, where nodes represent proteins annotated with abundance values, e.g. riBAQ values, and edges defined between nodes represent proteins that co-occur within the same sample;
2. **GNN representing the PPI networks** in which the proteins from the datasets are involved, likely utilizing STRING¹ database;
3. **GNN with adversarial training** inspired by GraphAT [24]. GraphAT has demonstrated promising results in enhancing robustness and generalization power to graph neural networks when exposed to adversarial examples. Extending GraphAT to link prediction tasks would require designing perturbations that affect not only node features but also edge features.

¹<https://string-db.org/>

Bibliography

- [1] R. Aebersold, J. N. Agar, I. J. Amster, M. S. Baker, C. R. Bertozzi, E. S. Boja, C. E. Costello, B. F. Cravatt, C. Fenselau, B. A. Garcia *et al.*, “How many human proteoforms are there?” *Nature chemical biology*, vol. 14, no. 3, pp. 206–214, 2018.
- [2] S. Al-Amrani, Z. Al-Jabri, A. Al-Zaabi, J. Alshekaili, and M. Al-Khabori, “Proteomics: Concepts and applications in human medicine,” *World journal of biological chemistry*, vol. 12, no. 5, p. 57, 2021.
- [3] E. F. Petricoin, K. C. Zoon, E. C. Kohn, J. C. Barrett, and L. A. Liotta, “Clinical proteomics: translating benchside promise into bedside reality,” *Nature reviews Drug discovery*, vol. 1, no. 9, pp. 683–695, 2002.
- [4] E. Gonçalves, R. C. Poulos, Z. Cai, S. Barthorpe, S. S. Manda, N. Lucas, A. Beck, D. Bucio-Noble, M. Dausmann, C. Hall *et al.*, “Pan-cancer proteomic map of 949 human cell lines,” *Cancer Cell*, vol. 40, no. 8, pp. 835–849, 2022.
- [5] J. Yoon, J. Jordon, and M. van der Schaar, “GAIN: missing data imputation using generative adversarial nets,” *CoRR*, vol. abs/1806.02920, 2018. [Online]. Available: <http://arxiv.org/abs/1806.02920>
- [6] J. Ribeiro. (2024) Synthetic missing data imputation in proteomics using generative adversarial networks. Accessed: January 03, 2025. [Online]. Available: <https://fenix.tecnico.ulisboa.pt/cursos/meaer21/dissertacao/1972678479056445>
- [7] S. J. Pan and Q. Yang, “A survey on transfer learning,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, Oct 2010.
- [8] C. Tan, F. Sun, T. Kong, W. Zhang, C. Yang, and C. Liu, “A survey on deep transfer learning,” *CoRR*, vol. abs/1808.01974, 2018. [Online]. Available: <http://arxiv.org/abs/1808.01974>
- [9] Y. Perez-Riverol, C. Bandla, D. J. Kundu, S. Kamatchinathan, J. Bai, S. Hewapathirana, N. S. John, A. Prakash, M. Walzer, S. Wang *et al.*, “The pride database at 20 years: 2025 update,” *Nucleic Acids Research*, vol. 53, no. D1, pp. D543–D553, 2025.

- [10] R. Aebersold and M. Mann, "Mass spectrometry-based proteomics," *Nature*, vol. 422, no. 6928, pp. 198–207, 2003.
- [11] —, "Mass-spectrometric exploration of proteome structure and function," *Nature*, vol. 537, no. 7620, pp. 347–355, 2016.
- [12] R. J. Little and D. B. Rubin, *Statistical analysis with missing data*. John Wiley & Sons, 2019, vol. 793.
- [13] T. Emmanuel, T. Maupong, D. Mpoeleng, T. Semong, B. Mphago, and O. Tabona, "A survey on missing data in machine learning," *Journal of Big data*, vol. 8, pp. 1–37, 2021.
- [14] W. Kong, H. W. H. Hui, H. Peng, and W. W. B. Goh, "Dealing with missing values in proteomics data," *Proteomics*, vol. 22, no. 23-24, p. 2200092, 2022.
- [15] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," *Advances in neural information processing systems*, vol. 27, 2014.
- [16] M. Zhao, Y. Cong, and L. Carin, "On leveraging pretrained gans for limited-data generation," *CoRR*, vol. abs/2002.11810, 2020. [Online]. Available: <https://arxiv.org/abs/2002.11810>
- [17] Y. Wang, C. Wu, L. Herranz, J. van de Weijer, A. Gonzalez-Garcia, and B. Raducanu, "Transferring gans: generating images from limited data," *CoRR*, vol. abs/1805.01677, 2018. [Online]. Available: <http://arxiv.org/abs/1805.01677>
- [18] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE transactions on neural networks*, vol. 20, no. 1, pp. 61–80, 2008.
- [19] L. Wu, P. Cui, J. Pei, and L. Zhao, *Graph Neural Networks: Foundations, Frontiers, and Applications*. Springer Nature, 2022.
- [20] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE transactions on neural networks and learning systems*, vol. 32, no. 1, pp. 4–24, 2020.
- [21] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.
- [22] L. Liu, Y. Ma, X. Zhu, Y. Yang, X. Hao, L. Wang, and J. Peng, "Integrating sequence and network information to enhance protein-protein interaction prediction using graph convolutional networks," in *2019 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. IEEE, 2019, pp. 1762–1768.

- [23] Z. Zhao, P. Qian, X. Yang, Z. Zeng, C. Guan, W. L. Tam, and X. Li, "Semignn-ppi: Self-ensembling multi-graph neural network for efficient and generalizable protein-protein interaction prediction," *arXiv preprint arXiv:2305.08316*, 2023.
- [24] F. Feng, X. He, J. Tang, and T.-S. Chua, "Graph adversarial training: Dynamically regularizing based on graph structure," *IEEE Transactions on Knowledge and Data Engineering*, vol. 33, no. 6, pp. 2493–2504, 2019.
- [25] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. Lempitsky, "Domain-adversarial training of neural networks," *arXiv preprint arXiv:1505.07818*, 2015.
- [26] S. Ben-David, J. Blitzer, K. Crammer, and F. Pereira, "Analysis of representations for domain adaptation," *Advances in neural information processing systems*, vol. 19, 2006.
- [27] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.



Additional Figures and Tables

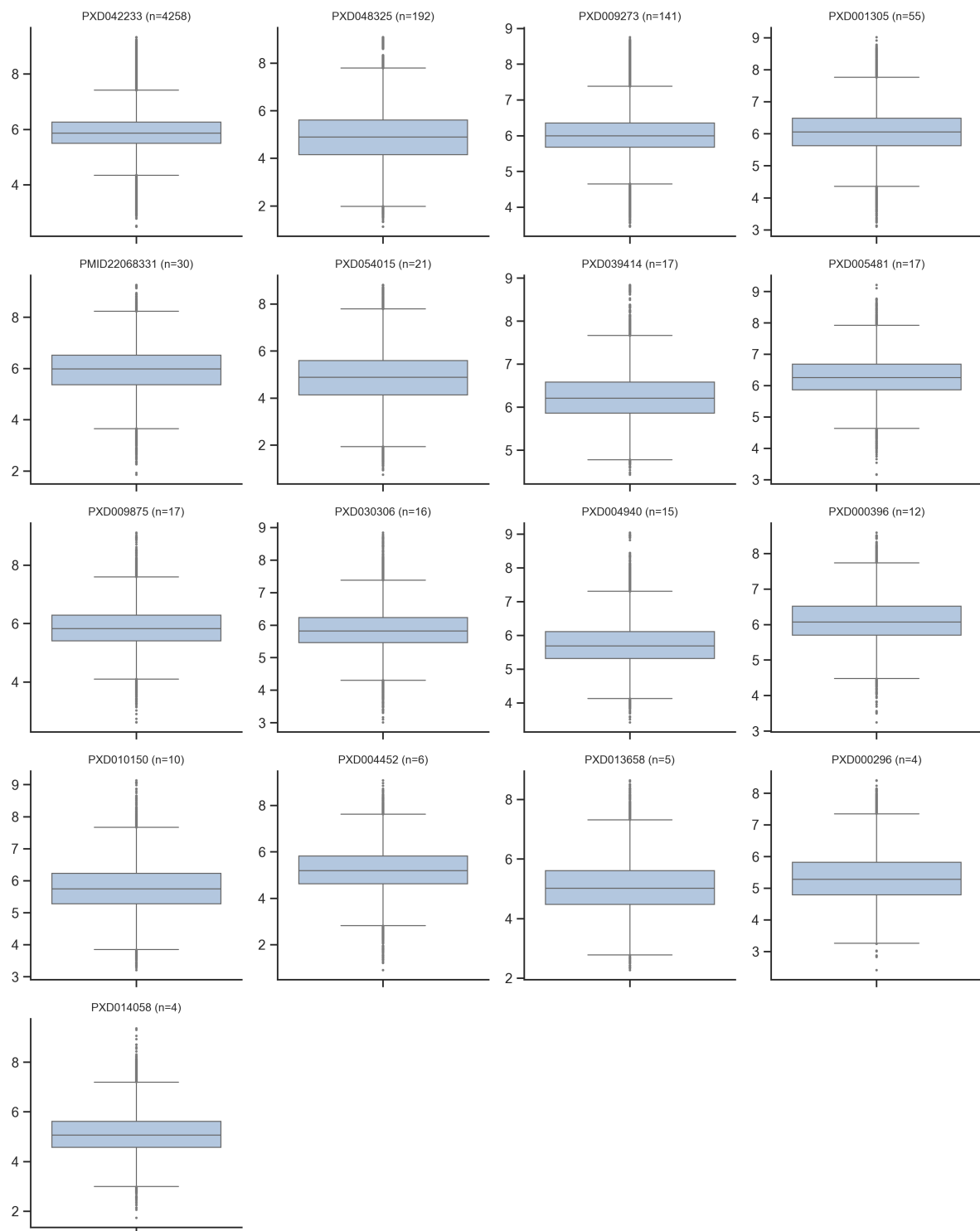


Figure A.1: Distribution of the protein intensities per project of the HeLa dataset. Each project is annotated with the number of samples indicated as (n= number of samples).

Project	Tissue	Number of Samples	Number of Proteins	Missing Rate (%)
PXD005736	Heart	2	8,470	2.10
PXD002179	Testis	3	8,836	3.38
PXD012131	Brain	26	10,188	6.77
PXD005445	Brain	7	9,165	7.42
PXD008840	Stomach	84	9,278	14.24
PXD010154	Liver, Lung, Colon, ...	38	14,669	17.15
PXD004452	Prostate, Colon, Liver	4	8,657	17.44
PXD019909	Skin	12	11,700	18.62
MSV000085836	Mixed, CAL-120, SK-HEP-1, ...	460	13,051	29.62
PXD006675	Heart	56	11,135	30.81
PXD020192	Frontal Cortex , Endometrium, Adrenal Gland, ...	46	6,177	34.88
PXD000612	HeLa-S3	11	8,782	39.62
PXD030304	HEK293T, SK-GT-2, SK-GT-4, ...	2,012	8,754	39.93
PXD000865	Placenta, Ovary, Prostate, ...	34	8,680	60.51
PXD000561	Heart, Liver, Ovary, ...	85	12,025	80.23

Table A.1: PRIDE datasets ordered by ascending missing rate. For datasets containing multiple tissue types, the three most represented tissues are listed. "..." indicates that additional tissue types are present. Tissue types in bold correspond to datasets containing a single tissue type.

